

Session 2.1

The Learning Problem: a Handwritten Digit, and the Neuron

What it means to learn a function from examples, and the smallest unit that does it

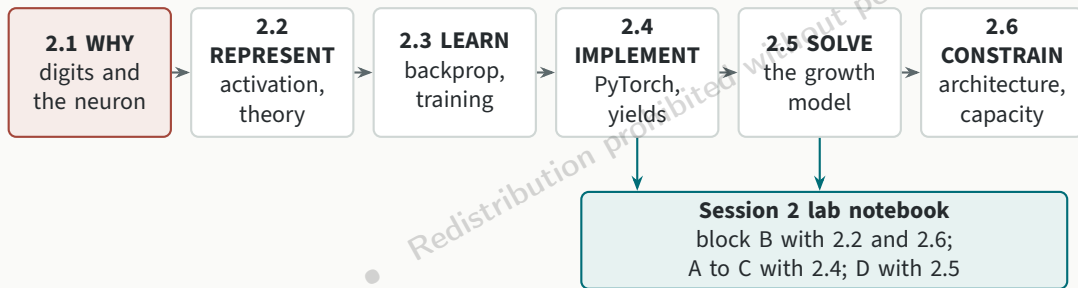


Zhigang Feng

What You Will Be Able to Do After Session 2

- ▶ **Read a network as a function:** write $f_{\theta}(x)$, count its parameters, and say what its architecture assumes about the input.
- ▶ **Compute one training step by hand:** forward pass, loss, backward pass, update.
- ▶ **Write the PyTorch training loop** and check it on a problem whose answer you already know.
- ▶ **Run an honest forecast comparison:** splits by date, one information set, a random-walk baseline.
- ▶ **Solve a growth model with a network**, and show with held-out tests that the solution is right.
- ▶ **Throughout: a small training loss is a claim about the training sample, and nothing more.**

Session 2 in One Map



- ▶ The running example for 2.1 to 2.3 is one picture: a handwritten digit.
- ▶ The economic applications are in 2.4 (a forecast) and 2.5 (a model solved).
- ▶ **One habit in all six decks: test a method first where the answer is known.**

What Session 1 Already Gave Us, and What It Only Named

- ▶ **Four verbs** for what AI does in economics: measure, predict, identify, compute (Session 1.2). Session 2 is about *predict* and *compute*.
- ▶ **Three kinds of learning**: supervised (inputs with labels), unsupervised (inputs only), reinforcement (learning by acting). Session 2 is supervised; reinforcement is Session 3.
- ▶ **Empirical risk minimisation**: choose θ to minimise the average loss $\frac{1}{n} \sum_i \ell(f_\theta(x_i), y_i)$ over a sample (Session 1.3). OLS is the linear, squared-loss case.
- ▶ Session 1 only *named* the rule that makes this honest: judge a model on data it never saw. Deck 2.3 turns it into a procedure, and every result after it is reported that way.
- ▶ **Today starts one step earlier: with a problem no formula solves.**

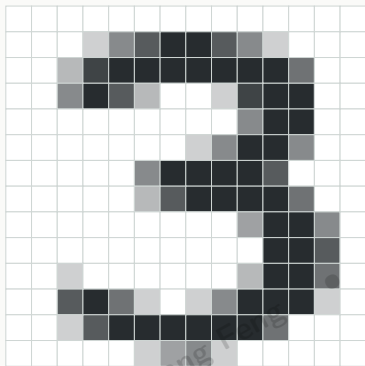
Session 2.1

A Handwritten Digit

One picture carries the ideas of Decks 2.1 to 2.3



A Handwritten Digit



- ▶ You read this as a “3” at once, and you cannot say how.
- ▶ The benchmark is **MNIST**: 70,000 digits written by US Census Bureau staff and high-school students, each a 28×28 grey-scale image with its label.
- ▶ Drawn here at 14×14 , so that every pixel can be seen.
- ▶ **Nobody writes that function down. It is learned from labelled examples.**

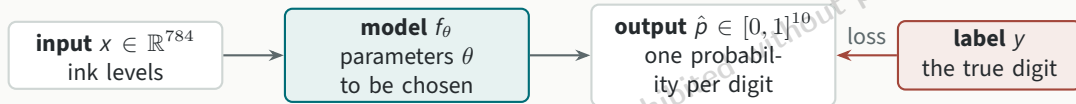
Source: Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, “Gradient-based learning applied to document recognition”, *Proc. IEEE* 86(11), 1998.

The Same Thing, Written Down

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	2	5	7	9	9	7	5	2	0	0	0	0
0	0	3	8	9	9	9	9	9	9	6	0	0	0	0
0	0	5	9	7	3	0	0	2	8	9	9	0	0	0
0	0	0	0	0	0	0	0	0	5	9	9	0	0	0
0	0	0	0	0	0	0	2	5	9	9	5	0	0	0
0	0	0	0	0	5	9	9	9	9	7	0	0	0	0
0	0	0	0	0	3	7	9	9	9	9	6	0	0	0
0	0	0	0	0	0	0	0	0	4	9	9	5	0	0
0	0	0	0	0	0	0	0	0	0	9	9	7	0	0
0	0	2	0	0	0	0	0	0	3	9	9	6	0	0
0	0	7	9	6	2	0	2	5	9	9	9	2	0	0
0	0	2	7	9	9	9	9	9	9	6	0	0	0	0
0	0	0	0	0	2	4	5	2	0	0	0	0	0	0

- ▶ To a computer the image is a table of ink levels: 0 is blank paper, 9 is full ink (0 to 255 in the original files).
- ▶ Read row after row, the table is a vector: 196 numbers here, $28 \times 28 = 784$ in MNIST. That vector is the entire input.
- ▶ Flattening forgets which pixels are neighbours. Deck 2.6 puts that back.
- ▶ **An image is a point in a space with one dimension per pixel: 196 here, 784 in MNIST.**

The Learning Problem: from 784 Numbers to Ten Classes



- **Data:** 60,000 labelled pairs (x_i, y_i) for training, 10,000 held out for testing.
- **Model:** a family of functions f_θ indexed by a parameter vector θ .
- **Loss:** small when $\hat{p}$ puts high probability on the true digit.
- **Learning:** choose θ to make the average training loss small, then measure on the held-out images.
- **The econometrician's problem with a richer f_θ : a specification, a criterion, an estimator.**

Try Writing the Rule for a “3”



four people, one digit

- ▶ A first rule: “two open curves, stacked”. Now define *curve*, *open* and *stacked* in ink levels.
- ▶ Slant, stroke width, size and position change all 784 numbers and none changes the digit.
- ▶ **Polanyi’s paradox:** we know more than we can tell.
- ▶ **When the rule cannot be written, give examples and let the rule be estimated.**

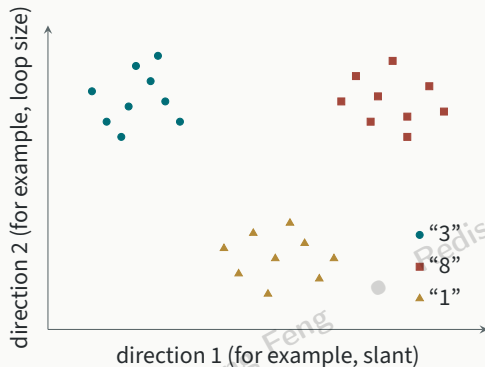
Source: M. Polanyi, *The Tacit Dimension*, 1966; D. Autor, “Polanyi’s paradox and the shape of employment growth”, NBER WP 20485, 2014.

Why It Is Possible at All: Structure

- ▶ There are 256^{784} possible 28×28 images. Almost all of them look like television static.
- ▶ Images of digits are a vanishing fraction of that space, and they are not scattered at random: strokes are continuous, connected and few.
- ▶ The **manifold hypothesis**: real high-dimensional data lie near a surface of much lower dimension inside the space they are recorded in.
- ▶ Macroeconomists use the same idea. Krusell and Smith (1998) replace an infinite-dimensional wealth distribution by its mean, because in equilibrium the distribution moves in very few directions.
- ▶ **Learning in high dimension works when the data have low-dimensional structure, and fails when they do not.**

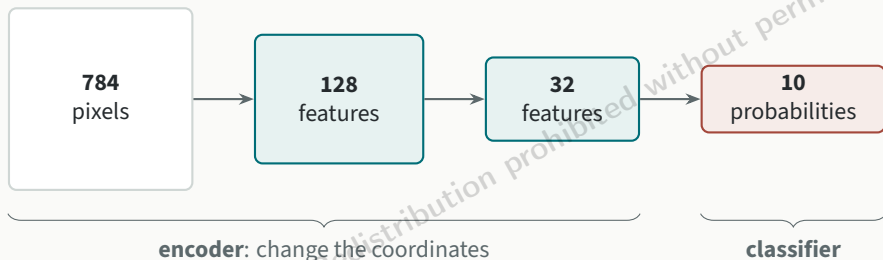
Source: P. Krusell and A. Smith, "Income and wealth heterogeneity in the macroeconomy", *JPE* 106(5), 1998.

From 784 Pixels to a Handful of Directions



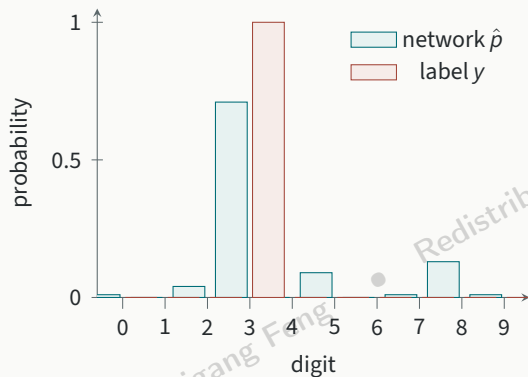
- ▶ Project the images onto two well-chosen directions and the digits form separate clouds (a schematic).
- ▶ A useful direction combines all 784 pixels. Principal components find such directions linearly; a network finds them nonlinearly.
- ▶ Once the clouds are apart, a line between them is enough.
- ▶ **Most of the work in learning is finding coordinates in which the problem is simple.**

The Encoder View



- ▶ Each layer re-describes the input in fewer, more useful numbers. The last layer is an ordinary multinomial logit on those numbers.
- ▶ The encoder and the classifier are estimated *jointly*, by the same criterion.
- ▶ In econometric terms: the regressors are estimated, not chosen by the researcher.
- ▶ **A deep network is a logit whose explanatory variables are learned.**

Labels, One-Hot Targets, and a Probability for Each Digit



- ▶ The label “3” is a **one-hot** vector: a 1 in position 3, zeros elsewhere.
- ▶ The **softmax** turns ten scores z_j into probabilities, $\hat{p}_j = e^{z_j} / \sum_k e^{z_k}$: McFadden’s multinomial logit, with scores as utilities.
- ▶ The loss is the negative log-likelihood of the true digit: $-\log 0.71 = 0.34$ here.
- ▶ **Classification is maximum likelihood for a discrete-choice model.**

What the Input's Structure Decides

Input	Structure it carries	Architecture	Economic example
image	neighbours matter	convolutional	night lights
sequence	order matters	recurrent, transformer	policy statements
set, network	order is irrelevant	graph network	households
state vector	none assumed	dense	(k, z) in a growth model

- ▶ Decks 2.1 to 2.5 use the dense network, which assumes nothing. Deck 2.6 returns to the other rows.
- ▶ **An architecture is an assumption about the input, written into the function.**

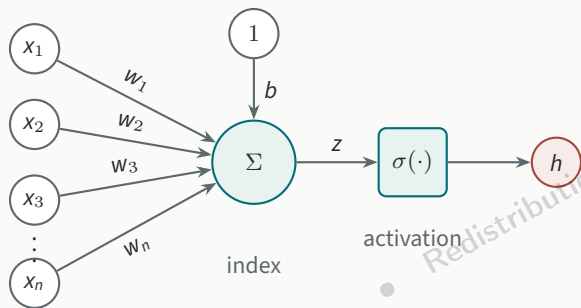
Session 2.1

The Neuron

A regression with a nonlinear output



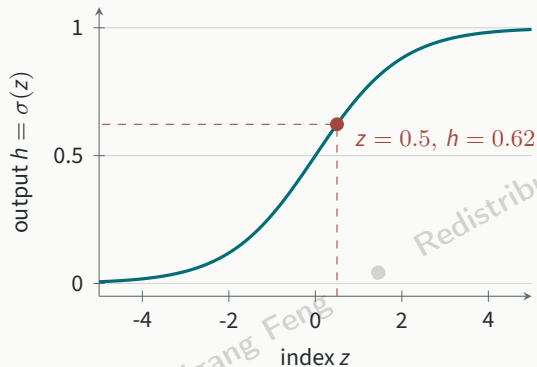
A Neuron Is a Regression with a Nonlinear Output



- ▶ Step 1, a linear index:
 $z = w^\top x + b$. The **weights** w are slopes; the **bias** b is the intercept.
- ▶ Step 2, a fixed nonlinear function, the **activation**: $h = \sigma(z)$.
- ▶ With $\sigma(z) = 1/(1 + e^{-z})$ this is exactly a logit model.
- ▶ The parameters are $\theta = (w, b)$: $n + 1$ numbers.
- ▶ **One neuron is a generalised linear model. Nothing more.**

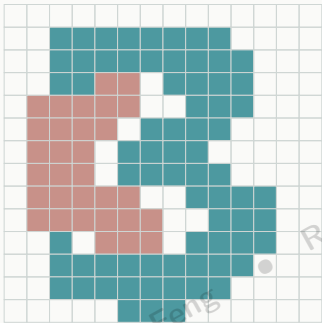
Source: Miao & Feng, *AI for Economists*, §9.1.

One Neuron, with Numbers



- ▶ Inputs $x = (1.0, 0.5, 0.0)$; weights $w = (0.8, -0.4, 0.3)$; bias $b = -0.1$.
- ▶ Index: $z = 0.8(1.0) - 0.4(0.5) + 0.3(0.0) - 0.1 = 0.5$.
- ▶ Output:
 $h = \sigma(0.5) = 1/(1 + e^{-0.5}) = 0.62$.
- ▶ Read it as a probability: the neuron is “62% on”.
- ▶ The third input is zero, so w_3 played no part. **A weight matters only where its input is active.**

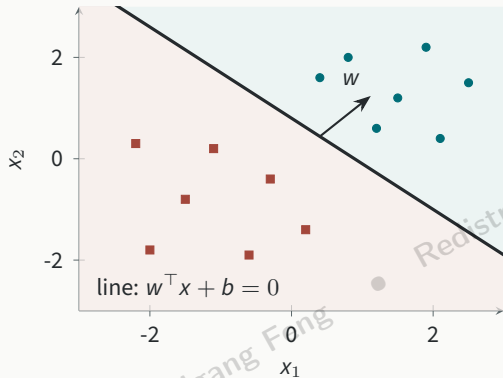
What the Weights Are: a Template Laid over the Image



teal: positive weight **red:** negative weight

- ▶ A neuron reading the image has one weight per pixel, so its weights can be drawn as an image.
- ▶ $w^T x$ is large when ink falls on positive weights and stays off negative ones.
- ▶ This one rewards ink where a “3” has strokes and penalises ink on the left, where an “8” closes its loops.
- ▶ **One neuron asks one question of the input: how much do you look like my template?**

One Neuron Draws One Line



- ▶ $\sigma(z) > 0.5$ exactly when $z = w^T x + b > 0$.
- ▶ The neuron switches on in a half-space whose edge is the **hyperplane** $w^T x + b = 0$: a line in two dimensions.
- ▶ w is perpendicular to the line; b slides it. The activation sets how sharply the output changes across the line, and does not bend it.
- ▶ **One neuron can separate only what a single straight cut can separate.**

An Economist Has Met This Before: Logit

- ▶ **Binary choice.** $\Pr(y = 1 \mid x) = \sigma(w^\top x + b)$ is the logit model; $w^\top x + b$ is the difference in systematic utility.
- ▶ **Estimation.** Maximum likelihood on the logit is minimising **cross-entropy**, the loss used to train classifiers. Same criterion, different name.
- ▶ **Vocabulary.** weights = coefficients; bias = intercept; activation = link function; training = estimation; inference = prediction.
- ▶ **What is new** is not the unit. It is stacking many units so that the regressors of the last one are themselves estimated.
- ▶ **Everything you know about specification, identification and out-of-sample testing still applies.**

Source: D. McFadden, “Conditional logit analysis of qualitative choice behavior”, in *Frontiers in Econometrics*, 1974.

Takeaways, and What Deck 2.2 Opens

- ▶ **A learning problem** is data, a family f_θ , a loss, and a rule for judging the result.
- ▶ **An image is a vector**, and learning works because real data occupy a small, structured part of the space.
- ▶ **A neuron is a logit**: a linear index through a fixed nonlinearity, $h = \sigma(w^\top x + b)$.
- ▶ **Alone it draws one line**, and one line is not enough for the digits or for XOR.
- ▶ **The whole of a network's flexibility sits in that σ** . We have used it without looking at it.
- ▶ **Next, Deck 2.2: what σ is, why a layer without it buys nothing, and what a network with it can reach.**

Session 2.2

Nonlinearity: the Activation, XOR, and What a Network Can Approximate

Why a layer without σ buys nothing, and which functions the family with it can reach



Zhigang Feng

What We Are Building, and What We Will Ask of It

- ▶ Deck 2.1 left one neuron, $h = \sigma(w^\top x + b)$, and one unexamined piece: the **activation** σ .
- ▶ **Question 1.** What is σ , and which ones will you actually meet?
- ▶ **Question 2.** Why is it needed at all — what fails in a layer without it, and what does a second layer fix that a wider first layer cannot?
- ▶ **Question 3.** How do kinks add up to a curve, and which functions can the family reach?
- ▶ Economists already choose flexible functional forms: translog, CES, polynomials, splines.
- ▶ **A network is one more flexible form. The difference is that it chooses its own basis.**

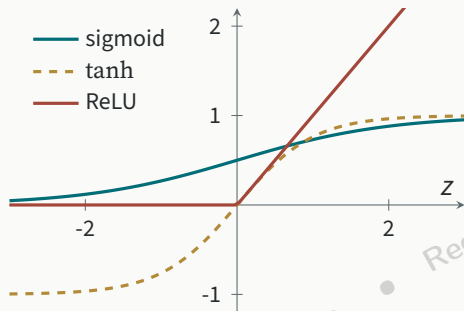
Session 2.2

The Activation

The one nonlinear piece, and the three you will meet



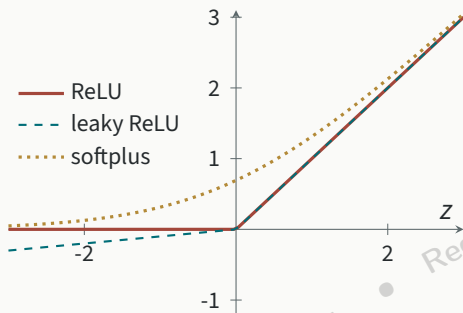
The Activations You Will Actually Meet



- ▶ **Sigmoid**, $1/(1 + e^{-z})$, in $(0, 1)$; **tanh**, in $(-1, 1)$. Both **saturate**: flat in the tails.
- ▶ **ReLU**, $\max(0, z)$: zero on one side, the identity on the other. Slope exactly 0 or 1.
- ▶ ReLU for hidden layers; sigmoid where an output must be bounded.
- ▶ **No activation on a regression output: it would silently bound the prediction.**

Source: Miao & Feng, *AI for Economists*, §9.2, Table 9.2.

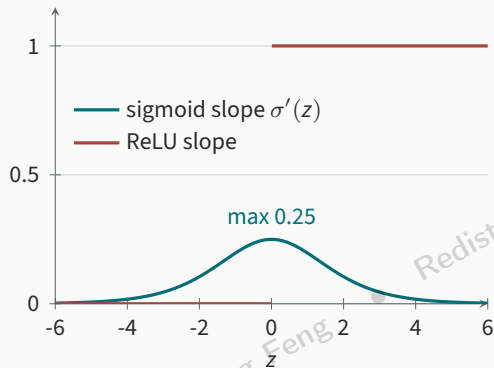
ReLU Is a Hinge



- ▶ $\max(0, z)$ is a call option's payoff, and an occasionally binding constraint: inactive, then linear.
- ▶ **Leaky ReLU**: a small slope on the flat side, so a unit that switched off can return.
- ▶ **Softplus**, $\log(1 + e^z)$, rounds the corner, for models that need second derivatives.
- ▶ **Each hidden ReLU unit contributes one hinge. A network is a sum of hinges.**

Source: Miao & Feng, *AI for Economists*, §9.2, Figs. 9.4–9.5.

Saturation, and Where the Vanishing Gradient Begins



- ▶ The sigmoid's slope is $\sigma(z)(1 - \sigma(z))$, never above 0.25.
- ▶ Training sends an error signal backwards through the layers (Deck 2.3), multiplied by this slope once per layer: at best 0.25^L .
- ▶ $L = 3$: 0.016. $L = 5$: 0.00098. $L = 7$: 6×10^{-5} . The early layers barely move: the **vanishing gradient**.
- ▶ **ReLU's slope is 1 on its active side, so the signal survives depth. That is why it is the default.**

Source: Miao & Feng, *AI for Economists*, §9.2.

Session 2.2

Why One Layer Is Not Enough

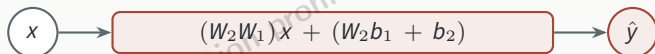
What collapses without σ , and what a hidden layer represents exactly



Linear Stacks Stay Linear



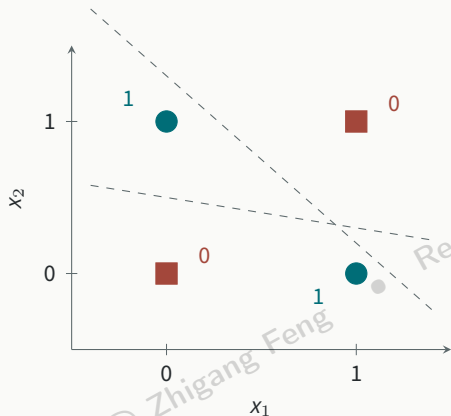
is the same function as



- ▶ Remove the activation and compose two layers:
 $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$.
- ▶ That is one linear map. A hundred linear layers are still one regression line.
- ▶ **Every bit of a network's flexibility comes from the activation between the linear maps.**

Source: Miao & Feng, *AI for Economists*, §9.1.

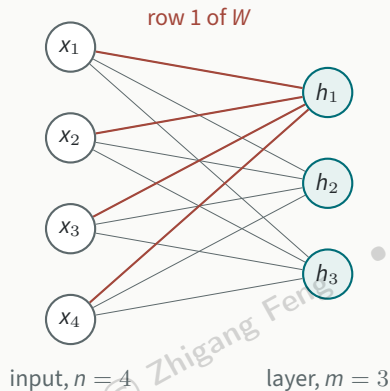
What One Neuron Cannot Do



- ▶ **XOR**, exclusive or: the output is 1 when exactly one of two binary inputs is 1.
- ▶ The two 1s sit on one diagonal, the two 0s on the other. No straight line separates them: try the dashed ones.
- ▶ Minsky and Papert (1969) made this the standard objection to single-layer networks.
- ▶ **The cure is a second layer. That is the rest of this section.**

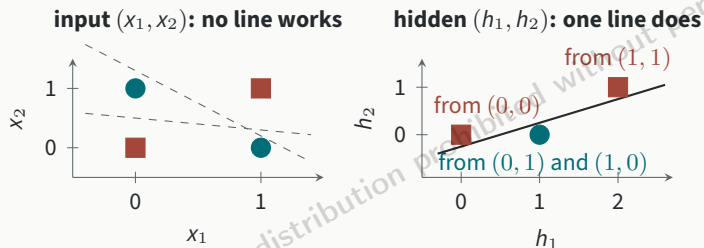
Source: M. Minsky and S. Papert, *Perceptrons*, MIT Press, 1969.

A Layer Is Many Neurons Reading the Same Input



- ▶ Put m neurons side by side. Each has its own weights and bias; all read the same x .
- ▶ Stack the weight vectors as rows of a matrix W ($m \times n$): $h = \sigma(Wx + b)$, with σ applied entry by entry.
- ▶ Here W is 3×4 : 12 weights and 3 biases, 15 parameters.
- ▶ **A layer is a system of m regressions sharing their regressors.**

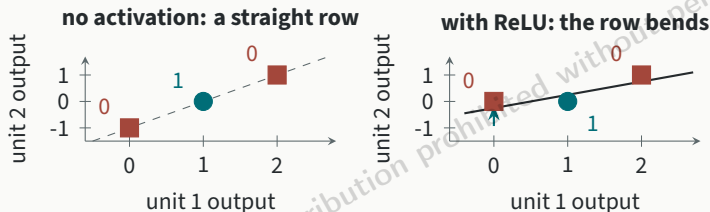
The Same Four Points, Moved by the Hidden Layer



- ▶ **The same four points, drawn twice.** Same marks, same colours; only the axes change.
- ▶ Write $s = x_1 + x_2$. Then $h_1 = \max(0, s)$ and $h_2 = \max(0, s - 1)$: $(0, 0) \mapsto (0, 0)$, both $(0, 1)$ and $(1, 0) \mapsto (1, 0)$, $(1, 1) \mapsto (2, 1)$.
- ▶ The drawn line is the output $\hat{y} = h_1 - 2h_2$: 0, 1, 1, 0, *exact*, not approximate. **The hidden layer did not fit a curve — it moved the points until one line was enough.**

Source: Miao & Feng, *AI for Economists*, §9.2; Goodfellow, Bengio, Courville, *Deep Learning*, 2016, §6.1.

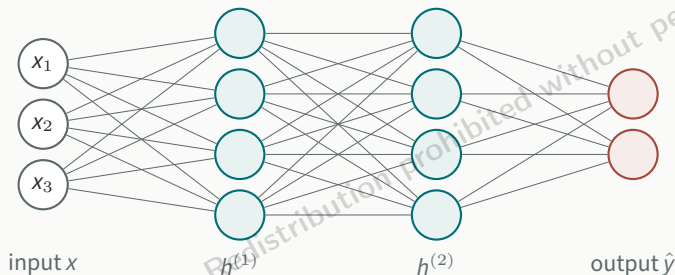
The Weights Move the Points; the Activation Bends Them



- ▶ Both units start from $s = x_1 + x_2$: one computes s , the other $s - 1$. **That step alone merges (0, 1) and (1, 0)**, since both have $s = 1$.
- ▶ **No activation**, left: the outputs are s and $s - 1$, so the points fall in a straight row, labelled 0, 1, 0. No cut splits the middle from the ends.
- ▶ **With ReLU**, right: $\max(0, s)$ and $\max(0, s - 1)$ raise that -1 to 0 , the row bends, and a cut works. **Moving and merging is the weights' work; bending is the activation's.**

Source: Arithmetic checked in the Session 2 lab notebook, block B.

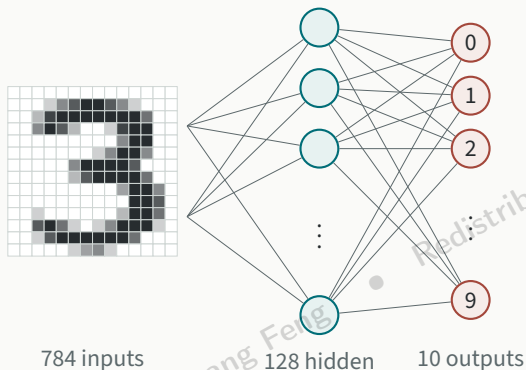
The Object in One Line



- ▶ A **feedforward network** (multilayer perceptron, MLP) composes layers:
$$f_{\theta}(x) = W_3 \sigma(W_2 \sigma(W_1 x + b_1) + b_2) + b_3.$$
- ▶ θ collects every W_{ℓ}, b_{ℓ} . **Width**: units per layer. **Depth**: number of layers.
- ▶ **A network is a parametric family: alternate a linear map with a fixed nonlinearity.**

Source: Miao & Feng, *AI for Economists*, §9.1, eq. (9.8).

A Network for Digits: 784 → Hidden → 10



- ▶ Layer 1: 784×128 weights + 128 biases = 100,480 parameters.
- ▶ Layer 2: $128 \times 10 + 10 = 1,290$. Total: **101,770**.
- ▶ Trained as in Deck 2.3, it classifies 97 to 98% of the 10,000 test images correctly.
- ▶ **More parameters than observations, and it still generalises. Deck 2.6 asks why.**

Source: Accuracy: the 784–128–10 example of the TensorFlow beginner tutorial; LeCun et al. (1998) report 95.3% with 1998 methods.

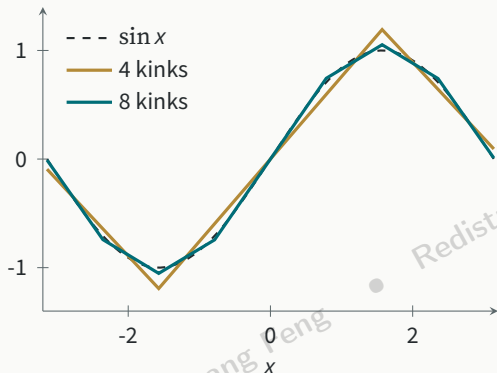
Session 2.2

How a Network Bends a Straight Line

Approximating $\sin x$ out of hinges, and letting the hinges move



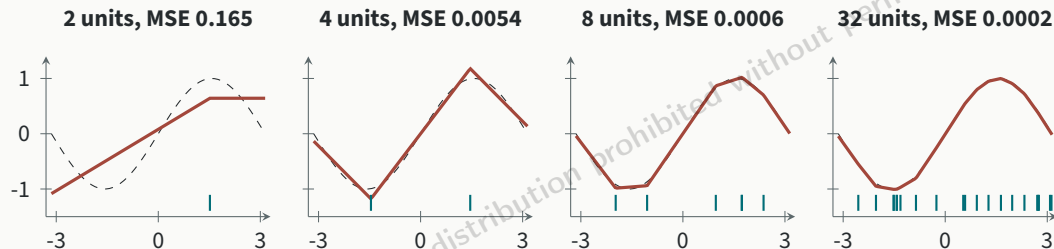
The Construction: an Approximation Built out of Kinks



- ▶ One hidden ReLU layer, one input:
 $\hat{y}(x) = a_0 + \sum_{i=1}^N a_i \max(0, x - b_i)$.
- ▶ The shifts b_i are the **kinks**, where the slope may change; the a_i set the slope on each interval.
- ▶ Fix the b_i , evenly spaced, and $\hat{y}$ is linear in the a_i : ordinary least squares on a piecewise-linear basis.
- ▶ **Mean squared error 0.0059 with 4 kinks, 0.0003 with 8. There is no network here yet.**

Source: Session 2 lab notebook, block B; least squares on 121 points of $\sin x$.

The Turn: Let the Kinks Choose Where to Go

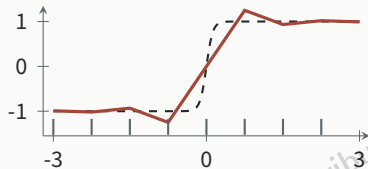


- ▶ Train the b_i together with the a_i : a one-hidden-layer network. The teal ticks mark the learned kinks, and they are not evenly spaced.
- ▶ At 8 units fixed kinks by least squares (0.0003) beat training (0.0006); at 4 units training (0.0054) beat fixed (0.0059). A search, not a solve.
- ▶ **The basis is no longer chosen in advance. The approximant decides where to bend.**

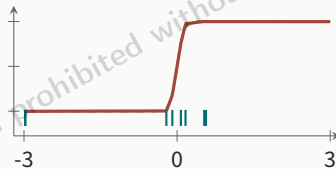
Source: Session 2 lab notebook, block B; Adam, 3,000 steps, one seed.

Where Adaptivity Pays, and Where It Cannot

8 kinks fixed, evenly spaced: MSE 0.0376



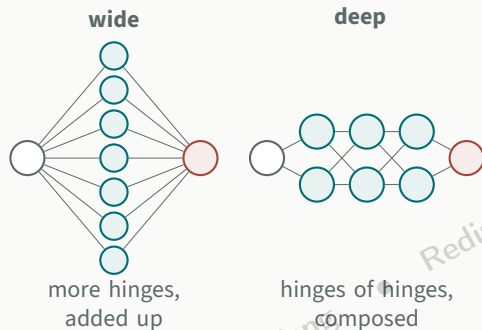
8 kinks learned: MSE 0.0001



- ▶ $\tanh(8x)$: all the curvature near zero. Eight units either way.
- ▶ Six of the eight learned kinks crowd into an eighth of the domain; the fit is **396 times** better. On $\sin x$, whose curvature is spread evenly, training lost.
- ▶ A fixed basis is linear in its parameters: a vector space, a closed-form solve, a rate quotable in advance. Width- m networks are none of these.
- ▶ **A solve becomes a search. What that buys is priced next.**

Source: Adam, 6,000 steps, one seed, against least squares on the same eight kinks. Session 2 lab notebook, block B.

Width and Depth: Two Axes



- ▶ Width adds basis functions; depth composes them.
- ▶ Cost: 20 inputs, three hidden layers of 256 units:
$$5,376 + 65,792 + 65,792 + 257 = 137,217$$
 parameters.
- ▶ Some functions need exponentially many units in two layers, polynomially many in three.
- ▶ **Depth pays when the target is a composition: data $\rightarrow$ beliefs $\rightarrow$ choice; households $\rightarrow$ aggregates.**

Source: R. Eldan and O. Shamir, "The power of depth for feedforward neural networks", COLT 2016; Miao & Feng, §9.1–9.2.

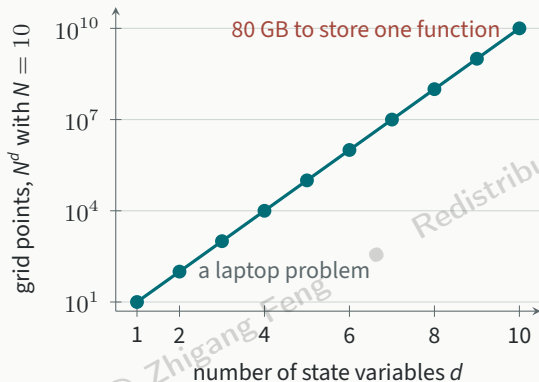
Session 2.2

What the Theory Says

Existence, rates, and what neither one promises



The Curse of Dimensionality, in Arithmetic



- ▶ A grid with N points per state variable needs N^d points in d dimensions.
- ▶ With $N = 10$: 100 points at $d = 2$, a million at $d = 6$, ten billion at $d = 10$, which is 80 GB for one policy function.
- ▶ Wealth, income, age, housing and two aggregate shocks already make $d = 6$.
- ▶ **A method whose cost grows with the number of examples, not with N^d , changes what can be solved.**

Universal Approximation, on a Compact Domain

- ▶ A **compact domain** is closed and bounded: $[0, 1]^d$, or a capital range $[k_{\min}, k_{\max}]$.
- ▶ **Universal approximation**: for any continuous f on a compact domain and any $\varepsilon > 0$ there is a one-hidden-layer network $\hat{f}(x) = \sum_{j=1}^m v_j \sigma(w_j^\top x + b_j) + c$ with $\sup_x |f(x) - \hat{f}(x)| < \varepsilon$.
- ▶ Cybenko (1989) for sigmoids; Hornik (1991) and Leshno et al. (1993) for any activation that is not a polynomial.
- ▶ Polynomials have the same property (Stone–Weierstrass), so universality does not distinguish a network.
- ▶ **The theorem says a good network exists on the domain. It says nothing outside the domain.**

Source: Cybenko, *Math. Control Signals Syst.* 2, 1989; Hornik, *Neural Networks* 4, 1991; Leshno, Lin, Pinkus, Schocken, *Neural Networks* 6, 1993.

Four Things the Theorem Does Not Say

- ▶ **Not how many units.** “There exists m ”, and m may be astronomically large.
- ▶ **Not how to find the weights.** Good weights exist; finding them is a non-convex search that promises only a stationary point.
- ▶ **Not how much data.** Approximating a known function is different from estimating an unknown one from a finite sample.
- ▶ **Not that it will generalise.** Fitting the sample and fitting the function are different achievements.
- ▶ **Finding the weights** is the binding one. With 8 kinks, least squares reached mean squared error 0.0003; training the same 8 units reached 0.0006. A better member of the family existed and the optimiser did not find it.
- ▶ **Existence is a statement about the family, not about the network you trained.**

Source: Miao & Feng, *AI for Economists*, §9.1.

Three Errors: Approximation, Estimation, Optimization



- **Approximation error:** the family cannot reach f . Falls with width and depth. The theorems live here.
- **Estimation error:** a finite sample picks the wrong member. Falls with data, rises with flexibility.
- **Optimization error:** the algorithm stops short of the sample optimum. Deck 2.3.
- **Only the sum is observed, on held-out data. A bigger network lowers the first error and can raise the other two.**

Source: L. Bottou and O. Bousquet, “The tradeoffs of large scale learning”, NeurIPS 2007.

Barron's Theorem, and the Class It Holds For

- ▶ Barron (1993): with m sigmoidal hidden units, on a ball of radius r ,

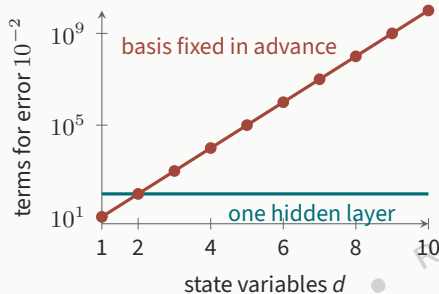
$$\int_{B_r} (f - \hat{f}_m)^2 d\mu \leq \frac{(2rC_f)^2}{m}, \quad C_f = \int \|\omega\| |\tilde{f}(\omega)| d\omega,$$

where $\tilde{f}$ is the Fourier transform of f .

- ▶ The **Barron class** is the set of functions with finite C_f : little weight at high frequencies.
- ▶ Double the units and the squared error halves, and the input dimension d is absent from the exponent.
- ▶ But C_f is an integral over $\mathbb{R}^d$. For many functions it grows with d ; for others it is infinite.
- ▶ **The dimension did not vanish. It moved into the constant and into the narrowness of the class.**

Source: A. Barron, "Universal approximation bounds for superpositions of a sigmoidal function", *IEEE Trans. Inf. Theory* 39, 1993.

What the Rate Costs: Terms Needed to Reach an Accuracy



- ▶ Terms to reach squared error ε : a fixed basis $m \sim \varepsilon^{-d/2}$, one hidden layer $m \sim (2rC_f)^2/\varepsilon$.
- ▶ d is in the exponent of the first and absent from the second. That is the content of $\leq C/m$.
- ▶ Each unit $\sigma(w^\top x + b)$ is a **ridge function**, flat along the $d - 1$ directions orthogonal to w : m one-dimensional profiles, never the whole box at once.
- ▶ **The curse is not removed. It moves into the hypothesis.**

Source: At $d = 1$ the fixed basis wins outright. Barron (1993), Props. 1 and 6.

The Rates, Side by Side

Functions	Approximator, size m	Best possible	Source
k times smooth	any continuous method	$\gtrsim m^{-k/d}$ (error)	DeVore et al. (1989)
Barron class	any basis fixed in advance	$\gtrsim m^{-2/d}$ (squared)	Barron (1993), Thm. 6
Barron class	one hidden layer	$\leq C/m$ (squared)	Barron (1993), Prop. 1
compositional	matching deep network	exponent free of d	Poggio et al. (2017)

- ▶ Rows 1–2 are the curse of dimensionality. Rows 3–4 escape it by assuming something about f .
- ▶ **The gap is between a basis fixed before seeing f and a basis adapted to f .**

Source: DeVore, Howard, Micchelli, *Manuscripta Math.* 63, 1989; Poggio et al., *Int. J. Autom. Comput.* 14, 2017.

Takeaways, and What Deck 2.3 Has to Supply

- ▶ **Without an activation, depth buys nothing.** With ReLU, each hidden unit is one hinge and a network is a sum of hinges.
- ▶ **A hidden layer changes the coordinates** until a line is enough; XOR shows this exactly, not approximately.
- ▶ **Fix the kinks and it is least squares; free them and the approximant chooses where to bend.**
- ▶ **Universal approximation is existence on a compact domain:** not size, not a recipe, not data, not generalisation.
- ▶ **Barron's rate has no d in the exponent;** the dimension lives in the constant and the class.
- ▶ **Every promise here was “a good network exists”. Deck 2.3 is how to find one, and how to know when to stop.**

Session 2.3

Forward Propagation, Backpropagation, and Training

How the parameters of a network are chosen, one step computed by hand, and when to stop



Zhigang Feng

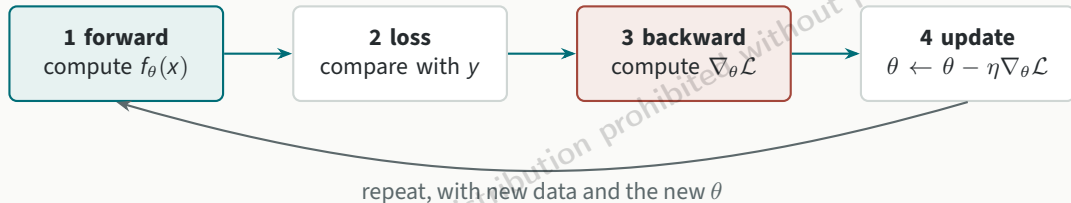
The Problem, Stated

- ▶ Deck 2.2 fixed a family of functions f_θ and showed that a good member exists. Training chooses θ :

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_\theta(x_i), y_i).$$

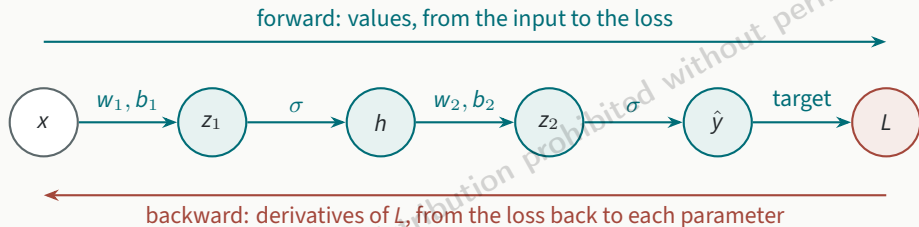
- ▶ The **loss** ℓ is a number that is small when the prediction $f_\theta(x_i)$ is close to the target y_i .
- ▶ θ has 10^3 to 10^9 entries, and $\mathcal{L}$ is not convex in θ . There is no formula for the minimiser.
- ▶ The only affordable information is the **gradient** $\nabla_{\theta} \mathcal{L}$: the direction in which the loss rises fastest.
- ▶ **Three questions organise this deck: how to get the gradient cheaply, what to do with it, and how to know the answer is worth anything.**

The Learning Loop: Forward, Loss, Backward, Update



- ▶ The **forward pass** evaluates the network, layer by layer, from input to prediction.
- ▶ The **backward pass** returns the derivative of the loss with respect to every parameter.
- ▶ The **learning rate** $\eta > 0$ is the step length: the one free choice in the update.
- ▶ **Every network in this course is trained by these four steps, repeated.**

One Graph, Two Traversals



- ▶ A **computation graph** records each intermediate value (node) and the operation behind it (edge).
- ▶ Backward walks the same edges in reverse, applying the **chain rule** edge by edge.
- ▶ The colour convention of this deck: **teal** and grey are values moving forward, **red** is derivatives moving back.
- ▶ **One graph, traversed twice. Backward reuses what forward stored.**

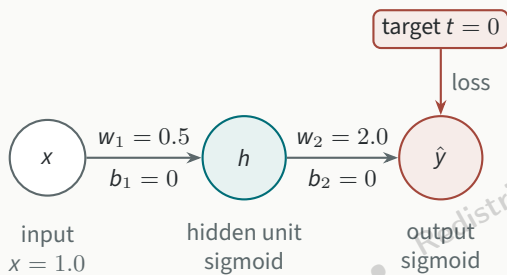
Session 2.3

Backpropagation, with Numbers

One observation, two layers, every derivative computed by hand

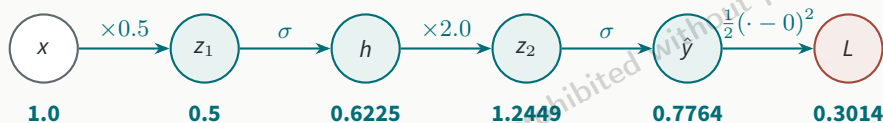


A Classifier, on One Observation



- ▶ The smallest honest version of the digit classifier: one input, one hidden unit, one output.
- ▶ $\hat{y} \in (0, 1)$ is the predicted probability that the image is a “3”. The target is $t = 0$: it is not.
- ▶ Loss: $L = \frac{1}{2}(\hat{y} - t)^2$, squared error, so that every step can be seen.
- ▶ **The arithmetic for one observation is the arithmetic for millions: forward, loss, backward, update.**

Forward, with Numbers

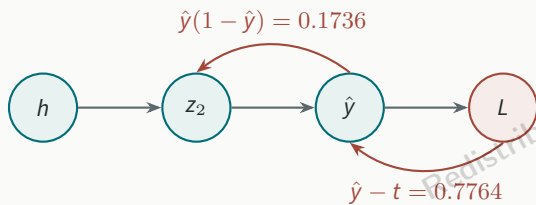


- ▶ $z_1 = w_1x + b_1 = 0.5$, and $h = \sigma(0.5) = 0.6225$: the hidden unit is “62% on”.
- ▶ $z_2 = w_2h + b_2 = 1.2449$, and $\hat{y} = \sigma(1.2449) = 0.7764$.
- ▶ $L = \frac{1}{2}(0.7764 - 0)^2 = 0.3014$. The network says “78% a 3”; the truth is that it is not.
- ▶ **Learning means moving w_1 and w_2 so that $\hat{y}$ falls. By how much each? That is the backward pass.**

Source: Session 2 lab notebook, block A2; every number is asserted there against PyTorch’s automatic differentiation.

Backward, Part One: the Output Layer

$$h = 0.6225$$

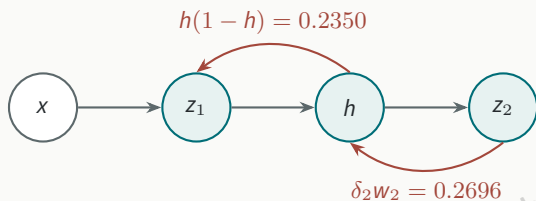


$$\delta_2 = \partial L / \partial z_2 = \mathbf{0.1348}$$

- ▶ Chain the links from w_2 to L :
$$\frac{\partial L}{\partial w_2} = \underbrace{(\hat{y} - t)}_{0.7764} \underbrace{\hat{y}(1 - \hat{y})}_{0.1736} \underbrace{h}_{0.6225}.$$
- ▶ Name the first two factors:
 $\delta_2 = \partial L / \partial z_2 = 0.1348$. It will be reused.
- ▶ Then $\partial L / \partial w_2 = \delta_2 h = 0.0839$ and $\partial L / \partial b_2 = \delta_2 = 0.1348$.
- ▶ **The sign is positive: raising w_2 raises the loss, so descent lowers w_2 .**

Source: Session 2 lab notebook, block A2.

Backward, Part Two: One Layer Further, and the Update

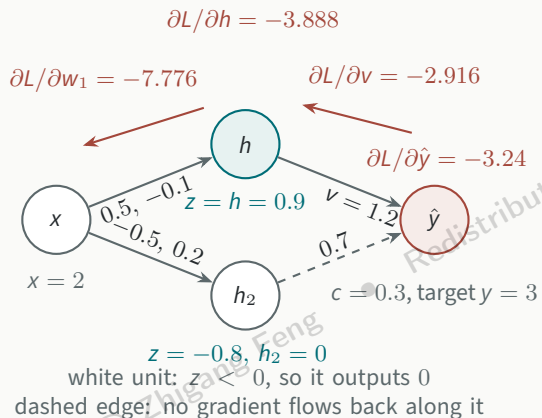


$$\delta_1 = \partial L / \partial z_1 = \mathbf{0.0633}$$

- ▶ Carry δ_2 back through w_2 :
 $\partial L / \partial h = \delta_2 w_2 = 0.2696$. Then through the activation:
 $\delta_1 = 0.2696 \times 0.2350 = 0.0633$.
- ▶ $\partial L / \partial w_1 = \delta_1 x = 0.0633$.
- ▶ Update with $\eta = 0.1$:
 $w_2 \leftarrow 2.0 - 0.1(0.0839) = 1.9916$ and
 $w_1 \leftarrow 0.5 - 0.1(0.0633) = 0.4937$.
- ▶ **Both sigmoid slopes are below 0.25. Each extra sigmoid layer shrinks the signal again: the vanishing gradient.**

Source: Session 2 lab notebook, block A2.

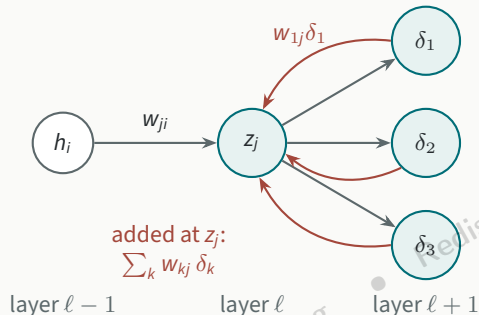
The Same Thing with ReLU — and a Unit That Gets No Gradient



- ▶ ReLU unit, $L = (\hat{y} - y)^2$. Forward: $\hat{y} = 1.2(0.9) + 0.3 = 1.38$, so $L = 2.6244$.
- ▶ Backward, in red: each derivative is the previous one times a local slope. ReLU's slope is 1: no shrinkage.
- ▶ $\eta = 0.01$: $w_1 \rightarrow 0.5778$, $v \rightarrow 1.2292$; the loss falls to 1.7488.
- ▶ **The second unit has $z < 0$: slope 0, gradient exactly 0. If that holds for every observation it never learns: a “dying ReLU”.**

Source: Miao & Feng, *AI for Economists*, §9.3.2; Session 2 lab notebook, block A2.

The Recursion, in General



- ▶ The **error signal** of a unit is $\delta_j = \partial L / \partial z_j$. At the output, $\delta = \hat{y} - y$ for squared error.
- ▶ **Back one layer:** add the messages from above, multiply by the local slope:
 $\delta_j = \sigma'(z_j) \sum_k w_{kj} \delta_k$.
- ▶ **A gradient:** $\partial L / \partial w_{ji} = \delta_j h_i$, signal at the weight's output end times activation at its input end.
- ▶ **Backpropagation is the chain rule with each intermediate derivative computed once and reused.**

Source: D. Rumelhart, G. Hinton, R. Williams, "Learning representations by back-propagating errors", *Nature* 323, 1986.

The Same Recursion, in Matrices

- ▶ Forward, for layers $\ell = 1, \dots, L$: $z^{(\ell)} = W_{\ell}h^{(\ell-1)} + b_{\ell}$ and $h^{(\ell)} = \sigma(z^{(\ell)})$, with $h^{(0)} = x$.
- ▶ Backward, from the top down ($\odot$ multiplies entry by entry):

$$\delta^{(\ell)} = (W_{\ell+1}^{\top} \delta^{(\ell+1)}) \odot \sigma'(z^{(\ell)}), \quad \frac{\partial L}{\partial W_{\ell}} = \delta^{(\ell)} (h^{(\ell-1)})^{\top}, \quad \frac{\partial L}{\partial b_{\ell}} = \delta^{(\ell)}.$$

- ▶ The backward pass uses the *transpose of the same matrix* that was used going forward.
- ▶ σ' multiplies at every layer, so the activation decides whether the signal survives depth.
- ▶ $h^{(\ell-1)}$ is needed on the way back, so forward activations must be stored: the memory cost of training.
- ▶ **Two lines of linear algebra per layer give every derivative in the network.**

Source: Miao & Feng, *AI for Economists*, §9.3, eqs. (9.20)–(9.21).

Backpropagation Is the Adjoint Method

- ▶ $\delta^{(\ell)}$ answers: by how much would the loss change if $z^{(\ell)}$ were perturbed? That is a **shadow price**.
- ▶ Treat each layer's equation $z^{(\ell)} = W_{\ell}h^{(\ell-1)} + b_{\ell}$ as a constraint. $\delta^{(\ell)}$ is its Lagrange multiplier.
- ▶ The backward recursion is a costate equation, solved from a terminal condition backwards, as in optimal control. There it is called the **adjoint method**; in numerical analysis, **reverse-mode differentiation**.
- ▶ A diagnosis follows: $\delta \approx 0$ in a layer means nothing below it can be improved at the current parameters. Printing gradient norms by layer is standard practice.
- ▶ **You already know how to read a multiplier. Backpropagation computes all of them in one backward sweep.**

Source: Miao & Feng, *AI for Economists*, §9.3.2; A. Griewank and A. Walther, *Evaluating Derivatives*, SIAM, 2008.

What Autograd Automates

- ▶ **Automatic differentiation** (autograd) builds the computation graph while the forward pass runs, then applies the recursion of the last three frames.
- ▶ The derivatives are exact to machine precision. Nothing is a finite difference, so there is no step size to choose and no truncation error to trade against rounding error.
- ▶ You write only the forward pass and the loss. No derivative is ever coded by hand.
- ▶ **Reverse mode** differentiates one scalar — the loss — with respect to all P parameters in a single backward sweep. That is exactly the shape of a training problem, and it is why the next frame's cost does not grow with P .
- ▶ **The forward pass is the only thing you write. Everything a gradient method needs follows from it.**

Source: Miao & Feng, *AI for Economists*, §9.3.4.

What a Gradient Costs

digits 784-128-10, $P = 101,770$, $n = 60,000$

 **1.90** × 187 → 355 ms

growth policy 1-32-32-1, $P = 1,153$, $n = 256$

 **3.33** × 0.11 → 0.36 ms

yield MLP 5-4-1, $P = 29$, $n = 300$

 **6.81** × 0.03 → 0.23 ms

Bar length: forward + backward per forward pass. Small networks: overhead, not arithmetic.

- ▶ Forward: about $2P$ operations per observation — a multiply and an add per weight. Backward returns two gradients per layer: for the weights, and for the layer's input.
- ▶ **All P derivatives at once cost a small multiple of one forward pass, whatever P is.** A numerical Jacobian needs $P + 1$ passes — $50,000 \times$ more at $P = 101,770$; a Hessian, P^2 numbers, impossible at $P = 10^6$.
- ▶ Textbooks say three passes. Measured: $1.59 \times$ with a leaf input, $2.90 \times$ when the input also needs a gradient; training never needs the first layer's, where nearly all the arithmetic is — about two.
- ▶ Budget $6PnE$: $P = 10^5$, $n = 5 \times 10^4$, $E = 100$ give 3×10^{12} operations, about 30 seconds on one core. Stored activations (batch $\times$ width $\times$ depth) cap the batch size, not statistics.
- ▶ **A macroeconomic network on a state vector is a small computation.**

Source: Measured on one core of an AMD EPYC 7763; Session 2 lab notebook, block A5. A. Griewank and A. Walther, *Evaluating Derivatives*, SIAM, 2008; J. Nocedal and S. Wright, *Numerical Optimization*, 2nd ed., 2006; J. Kaplan et al., arXiv:2001.08361, 2020.

Session 2.3

The Loss, and the Optimizer

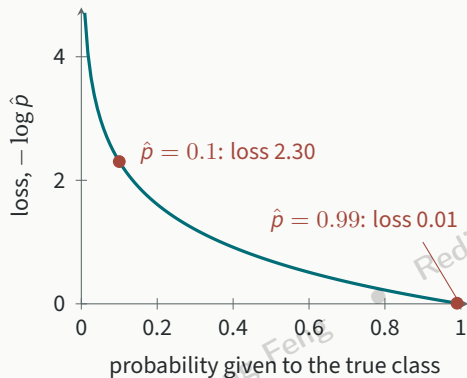
What the criterion assumes, and what to do with a gradient that is noisy



Loss I: Squared Error Is OLS

- ▶ For a continuous target, the mean squared error: $\mathcal{L} = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$.
- ▶ Minimising it is maximum likelihood under Gaussian errors. For a linear f_θ it is ordinary least squares.
- ▶ A loss is therefore an assumption about the noise, not a convention. Change the noise model and the loss changes: Poisson likelihood for counts, the check function for quantiles.
- ▶ In Deck 2.4 the target is next month's yield. In Deck 2.5 the "prediction" is an Euler-equation residual and the target is zero, so no data enter the loss at all.
- ▶ **Squared error asks for the conditional mean. If that is not what you want, it is the wrong loss.**

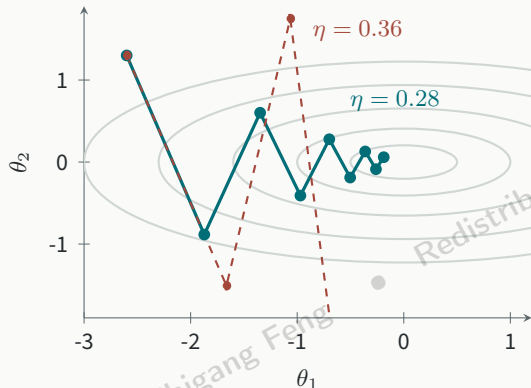
Loss II: Cross-Entropy Is McFadden



- ▶ A classifier returns a score z_k for each of K classes. The **softmax** turns scores into probabilities: $\hat{p}_k = e^{z_k} / \sum_j e^{z_j}$.
- ▶ **Cross-entropy** is the negative log-probability given to the true class, $\mathcal{L} = -\log \hat{p}_{k^*}$.
- ▶ Softmax is the logit choice probability, the label is the observed choice, and minimising cross-entropy maximises McFadden's multinomial-logit likelihood.
- ▶ **Confident and right costs nothing. Confident and wrong is unbounded.**

Source: D. McFadden, "Conditional logit analysis of qualitative choice behavior", in *Frontiers in Econometrics*, 1974.

Gradient Descent: the Engine

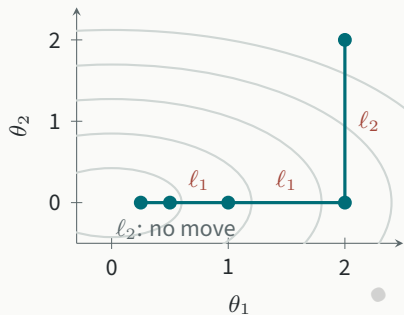


► Gradient descent:

$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$. Step against the gradient.

- The picture is $\mathcal{L} = \frac{1}{2}(\theta_1^2 + 6\theta_2^2)$. The teal path zigzags across the steep direction and converges.
- The red path uses a step only slightly longer. It overshoots further each time and diverges.
- **The learning rate is limited by the steepest direction, and progress by the flattest.**

Why a Sampled Gradient Still Gets There



- ▶ **Stochastic gradient descent** (SGD): differentiate a random subset of the loss's terms at each step.
- ▶ Here $\ell_1 = \frac{1}{2}\theta_1^2$, $\ell_2 = \theta_2^2$, step 0.5: only the drawn coordinate moves.
- ▶ Unbiased: its expectation is the full gradient.
- ▶ The surface is not convex, but with far more parameters than needed, most local minima are nearly as deep.
- ▶ **The path is not monotone and is not meant to be — and the goal is not the lowest training loss but a minimum that generalises.**

Source: L. Bottou, F. Curtis, J. Nocedal, *SIAM Rev.* 60(2), 2018; A. Choromanska et al., AISTATS 2015; T. Garipov et al., NeurIPS 2018.

Batch, Stochastic, Mini-Batch

Method	Terms per step	Cost of a step	Path
full batch	all n	high	smooth
stochastic	1	lowest	very noisy
mini-batch	32 to 1,024	one matrix product	noisy, averaged

- ▶ An **epoch** is one pass through the training data; an **iteration** is one update. With 40,000 observations and batches of 256, an epoch is 156 updates.
- ▶ Why mini-batch: many updates per pass, activations that fit in memory, hardware built for matrix products.
- ▶ The gradient noise also helps the iterate leave saddle points and shallow minima.
- ▶ **Count updates, not epochs: 100 epochs is 15,600 updates here and 600 on a sample of 1,600.**

Convergence: What Is Actually Guaranteed

- ▶ If the gradient is Lipschitz, its sampling noise has bounded variance, and the step sizes satisfy the **Robbins–Monro conditions**

$$\sum_n \eta_n = \infty \quad \text{and} \quad \sum_n \eta_n^2 < \infty,$$

then the expected squared gradient along the path goes to zero.

- ▶ $\sum \eta_n = \infty$, “go the distance”: the iterate can travel any distance from a bad start.
- ▶ $\sum \eta_n^2 < \infty$, “kill the noise”: accumulated sampling noise stays finite, so late steps stop bouncing.
- ▶ $\eta_n = 1/n$ satisfies both. A constant η fails the second: SGD then hovers in a ball around a solution.
- ▶ **The promise is a stationary point. Not a global minimum, not a rate, not a bound on the error.**

Source: H. Robbins and S. Monro, “A stochastic approximation method”, *Ann. Math. Statist.* 22(3), 1951; Bottou, Curtis, Nocedal (2018), §4.

Adaptive Step Sizes: RProp → RMSProp → Adam

- ▶ **Momentum**: step along an exponentially weighted average of past gradients. An economist would call it adaptive expectations about the descent direction.
- ▶ **RProp** (1993) sized each parameter's step from the sign of its gradient. **RMSProp** (2012) made that work with mini-batches: divide each step by a running average of that parameter's squared gradients.
- ▶ **Adam** (2015) is momentum plus RMSProp, with decay rates 0.9 and 0.999 by default.
- ▶ **AdamW** (2019) applies **weight decay**, a shrinkage of the weights at each step, outside the adaptive scaling. It is today's default. Later proposals: Lion, Sophia, schedule-free methods.
- ▶ **Adam is SGD with a step size for each parameter. It is still first-order: no Hessian is formed or stored.**

Source: D. Kingma and J. Ba, "Adam", ICLR 2015; I. Loshchilov and F. Hutter, "Decoupled weight decay regularization", ICLR 2019; M. Riedmiller and H. Braun, IEEE ICNN 1993.

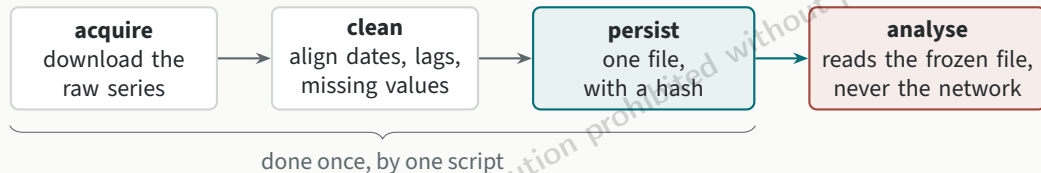
Session 2.3

The Dataset, and Three Samples

Where the data come from, and the rule that makes a reported number mean something



The Data Workflow: Acquire, Clean, Persist — Once



- ▶ Statistical agencies revise. A notebook that downloads on every run gives different answers on different days.
- ▶ Separate the download from the analysis; the analysis then runs offline and gives the same numbers every time.
- ▶ **A result is reproducible only if its data are frozen.**

Train, Validation, Test: Three Jobs, Three Samples

training sample: choose the parameters θ

the optimiser sees only this

validation

choose the design:
width, penalty,
when to stop

test

report once,
at the end

- ▶ A flexible model can fit its training sample as closely as you like. That says nothing about new data.
- ▶ **Validation** data choose between designs, so they are no longer a clean test.
- ▶ **Test** data are touched once. Their error estimates the error on future data.
- ▶ With time series the three samples are consecutive periods, never a shuffle.
- ▶ **Each sample does one job. A sample used for two jobs does neither honestly.**

Leakage: Four Ways the Test Set Gets into Training

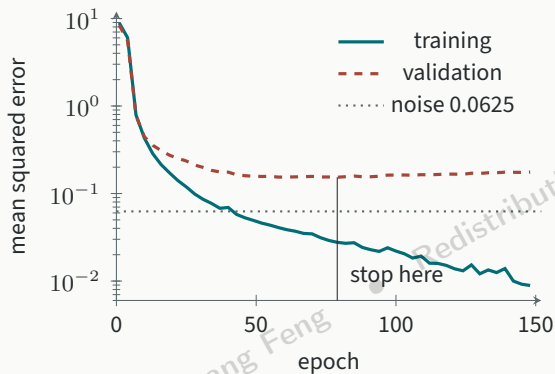
- ▶ **Leakage:** information from the test sample reaches the fitted model, so the test error is too good.
- ▶ **Shuffling a time series.** Neighbouring months are nearly identical; a shuffled split puts the answer for March next to the question for April.
- ▶ **Scaling with full-sample statistics.** The mean and standard deviation used to standardise the inputs must come from the training window alone.
- ▶ **Tuning on the test set.** Trying ten designs and reporting the best test error makes the test set a validation set.
- ▶ **The same template on both sides.** In the lab, sentences built as subject $\times$ phrase score 0.97 on a random split and 0.50 when whole phrases are held out.
- ▶ **Ask of every reported accuracy: what did the model already know about these cases?**

Source: Accuracies: Session 2 lab notebook, appendix (logistic regression on word counts, 96 sentences).

Validation Is the Part You Cannot Delegate

- ▶ An AI assistant will write every line of Deck 2.4's code, correctly, in seconds.
- ▶ It cannot tell you whether the result is *right*, because it does not know what right means for your problem: which benchmark, which tolerance, which sample is held out.
- ▶ **You supply:** the question, the model, the information set, the accuracy criterion.
- ▶ **The tool supplies:** syntax, the translation into code, the routines, the plots.
- ▶ A training loop that runs without error has told you nothing yet. A training loop that reproduces a known answer has told you the loop is right.
- ▶ **The bottleneck moved from writing the loop to saying what the loop must satisfy.**

Overfitting, Underfitting, and When to Stop



- **Underfitting:** both curves high.
- **Overfitting:** training loss falls while validation loss turns up.
- $n = 2,000$. Training loss ends at 0.0085, below the noise variance 0.0625: memorised noise.
- **Early stopping:** keep the best-validation parameters, near epoch 80.
- **A training loss below the noise floor is a diagnosis, not an achievement.**

Source: Recorded in the instructor's quantitative-macro course computations (2026); not recomputed in this session's lab.

Three Instruments Against Overfitting, and What They Are Worth

Strategy	best validation	
	$n = 2,000$	$n = 50,000$
none	0.1535	0.0788
weight decay 10^{-4}	0.1536	0.0804
dropout 0.1	0.1257	0.0780
dropout 0.2	0.1341	0.0792
both (0.1)	0.1238	0.0780

Noise floor 0.0625; 30 inputs, 17 pure noise.

Source: Recorded in the instructor's quantitative-macro course computations (2026), median of three seeds; not recomputed in this session's lab.

- ▶ **Weight decay:** a penalty $\lambda \|\theta\|^2$, which is ridge regression.
- ▶ **Dropout:** set a random share of units to zero at each training step. It averages many sub-networks.
- ▶ Early stopping acts like a ridge penalty of about $1/(\eta T)$ after T steps.
- ▶ **Dropout pays where data are scarce: 18% at $n = 2,000$, 1% at $n = 50,000$. More data does more: 0.1535 to 0.0788.**

Takeaways, and What Is Still Missing

- ▶ **Backpropagation** is the chain rule with intermediate derivatives named and reused. Those derivatives are shadow prices.
- ▶ **The optimiser is first-order because it must be:** a Hessian at $P = 10^6$ has 10^{12} entries. Adam is SGD with a step size for each parameter.
- ▶ **What is guaranteed** is a stationary point under the Robbins–Monro conditions, and nothing stronger.
- ▶ **Each sample does one job:** the optimiser sees the training data, the design is chosen on validation data, and the test data are touched once.
- ▶ **Generalisation is measured, not assumed:** a training loss below the noise floor is the mark of overfitting, and regularisation pays mainly when data are scarce. Deck 2.6 returns with a case the noise floor cannot diagnose at all.
- ▶ **Next, Deck 2.4: the same loop in PyTorch, first where the answer is known, then on next month's 10-year yield.**

Session 2.4

PyTorch, Curve Fitting, and Yield Forecasts

The training loop in code, a curve fitted and broken, and an honest one-month-ahead forecast



Zhigang Feng

What PyTorch Actually Is

tensors

arrays of numbers, as in NumPy

autograd

a record of every operation, so exact derivatives come back

nn.Module

a class that holds a network's parameters

optim

gradient-descent variants: SGD, Adam

- ▶ A scientific-computing library that became popular for deep learning, not the reverse.
- ▶ The first two pieces are the substance: write any scalar as a chain of array operations and its gradient is free.
- ▶ The last two are bookkeeping for parameters and update rules.
- ▶ **Everything in Deck 2.3 is four library calls. Today we make them.**

Source: A. Paszke et al., "PyTorch: an imperative style, high-performance deep learning library", *NeurIPS* 32, 2019.

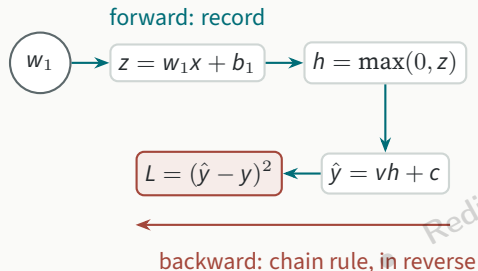
Tensors: Shapes, Types, and the Broadcasting Bug

```
k = torch.linspace(0.5, 2.0, 4)      # shape (4,)
K = k.reshape(-1, 1)                 # shape (4, 1): four observations, one feature
print(k.shape, K.shape, K.dtype)     # torch.Size([4]) torch.Size([4, 1]) torch.float32
print((K - k).shape)                 # torch.Size([4, 4]) <- not what you meant
```

- ▶ A **tensor** is an array that knows its **shape** (here 4 rows, 1 column) and its **dtype**, the number format (32-bit floats by default).
- ▶ **Broadcasting** stretches arrays of different shapes to match. A (4, 1) minus a (4,) silently becomes a 4×4 table, and the loss that follows is wrong without any error message.
- ▶ Convention in this course: a batch of n observations of d variables has shape (n, d) , always.
- ▶ **Most first bugs are shape bugs. Print the shape before you trust the number.**

Source: Session 2 lab notebook, block A1.

Autograd: the Tape



- ▶ Mark a tensor `requires_grad=True` and PyTorch records every operation it enters: the **computation tape**.
- ▶ `backward()` walks the tape in reverse, one chain-rule step per operation: an exact derivative, not a finite difference.
- ▶ This is **autograd**: Deck 2.3's backward pass, done by the library.
- ▶ **You write the forward pass only.**

backward(): Deck 2.3's Numbers, Reproduced

```
x, y = 2.0, 3.0
w1, b1, v, c = (torch.tensor(a, requires_grad=True) for a in (0.5, -0.1, 1.2, 0.3))
z = w1 * x + b1; h = torch.relu(z); yhat = v * h + c; L = (yhat - y) ** 2
L.backward()
# forward : z = 0.9000, h = 0.9000, yhat = 1.3800, L = 2.6244
# backward: dL/dv = -2.9160, dL/dc = -3.2400, dL/dw1 = -7.7760, dL/db1 = -3.8880
# update   : w1 -> 0.5778, v -> 1.2292; loss 2.6244 -> 1.7488      (eta = 0.01)
```

- ▶ The four-parameter ReLU example of Deck 2.3, in four lines. Every number on those slides is asserted against autograd in the lab.
- ▶ A second hidden unit with $z = -0.8 < 0$ receives a gradient of exactly 0.0: no signal, no learning.
- ▶ **If your hand derivative and autograd disagree, the hand derivative is wrong.**

Source: Session 2 lab notebook, block A2; example from Miao & Feng, *AI for Economists*, §9.3.

The Training Loop Is Always the Same Five Lines

```
model = nn.Linear(2, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
for step in range(300):
    prediction = model(X)                # 1 forward
    loss = ((prediction - y) ** 2).mean() # 2 loss
    optimizer.zero_grad()                # 3 clear the old gradients
    loss.backward()                      # 4 backward
    optimizer.step()                     # 5 update
```

- ▶ Forward, loss, clear, backward, update. A language model with billions of parameters is trained by these five lines.
- ▶ Only line 1 (the model) and line 2 (the loss) change between applications.
- ▶ Fit a curve: L is the squared prediction error. Solve a model: L is the squared Euler residual, which is Deck 2.5.
- ▶ **Once derivatives are free, one algorithm covers both. Learn the loop once.**

Source: Session 2 lab notebook, block A3.

A Network Is a Class

```
class SimpleNet(nn.Module):
    def __init__(self, hidden=32, act=nn.ReLU):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(1, hidden), act(), nn.Linear(hidden, 1))
    def forward(self, x):
        return self.net(x)
```

- ▶ `nn.Module` is PyTorch's base class for anything with parameters. `__init__` declares the layers; `forward` says how an input flows through them.
- ▶ `nn.Linear(1, hidden)` is $WX + b$ with W of shape $\text{hidden} \times 1$; `nn.Sequential` composes layers in order.
- ▶ This is $f_{\theta}(x) = W_2 \sigma(W_1 x + b_1) + b_2$ from Deck 2.2, with 32 hidden units.
- ▶ **The class is the specification. Read it as you would read a regression equation.**

Source: Session 2 lab notebook, block A4.

Optimizers, zero_grad, and Seeds

```
torch.manual_seed(1); net = SimpleNet()           # same seed, same initial draw
optimizer = torch.optim.Adam(net.parameters(), lr=3e-3, weight_decay=weight_decay)
optimizer.zero_grad()                             # 3 clear the old gradients
loss = loss_fn(net(X_tr), y_tr); loss.backward(); optimizer.step()
net.eval()                                         # evaluation mode
with torch.no_grad():                             # no tape: nothing to differentiate
    val = float(loss_fn(net(X_va), y_va))
```

- ▶ `parameters()` hands every W and b to the **optimizer**. `weight_decay` is an L2 penalty on the weights: ridge regression, in one argument.
- ▶ Gradients *accumulate* across calls to `backward()`: without `zero_grad()` you descend on the sum of every gradient so far.
- ▶ Initial weights are a random draw and a **seed** fixes it. `eval()` switches layers such as dropout; `no_grad()` turns the tape off when you only want predictions.
- ▶ **Forgetting `zero_grad()` raises no error. It trains the wrong model.**

Source: Session 2 lab notebook, blocks A4 and C.

Start at Version 0, Where You Know the Answer

```
torch.manual_seed(0)
X = torch.randn(200, 2)
y = 1.0 + X @ torch.tensor([[2.0], [-0.5]]) + 0.1 * torch.randn(200, 1)
# ... the five-line loop on nn.Linear(2, 1), 300 steps ...
# OLS      : [ 1.0073  1.9972 -0.5023]
# learned: [ 1.0073  1.9972 -0.5023]
assert np.allclose(ols, learned, atol=1e-3)
```

- ▶ A linear “network” trained on squared error must land on the OLS coefficients. It does, to four decimals.
- ▶ If it had not, the bug would be in the loop, and you would know before touching a real problem.
- ▶ The same habit in Deck 2.5: solve the growth model first where the policy is known in closed form.
- ▶ **Version 0 of any project is the case with a known answer.**

Source: Session 2 lab notebook, block A3.

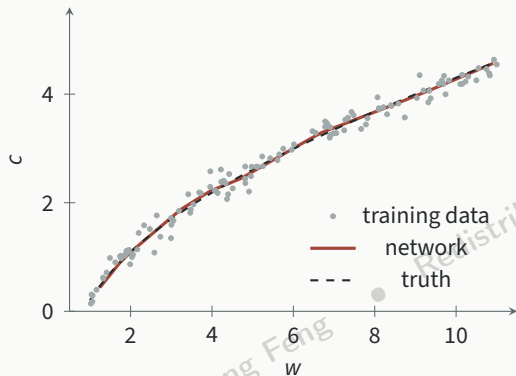
Session 2.4

A Curve Whose Answer Is Known

Fit it, read the residuals, then ask it about a place it has never been



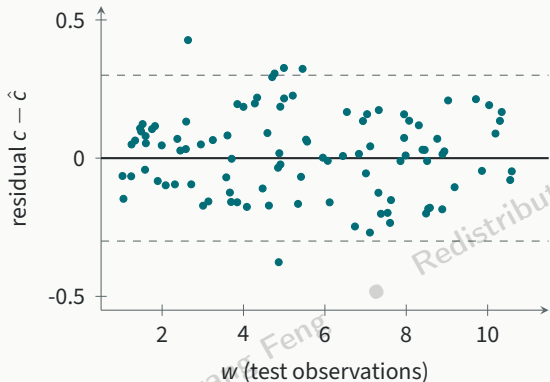
Fit: a Network Finds $c = f(w)$



- ▶ A hidden law, $c = \ln w + 0.2 w + \varepsilon$ on $w \in [1, 11]$, noise variance 0.0225; 300 training and 100 test observations.
- ▶ 32 ReLU units, 1,500 Adam steps. The functional form was never specified.
- ▶ Mean squared error: 0.0231 in training, 0.0229 on the test data.
- ▶ **Both sit at the noise variance: the law was found, the noise was not chased.**

Source: Session 2 lab notebook, block B.

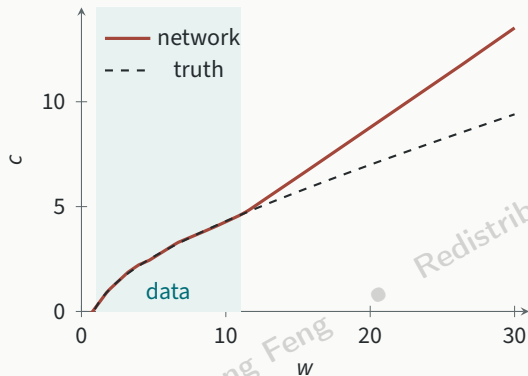
Residuals: What Is Left Should Have No Shape



- ▶ The test residuals scatter evenly around zero across the whole range of w . The dashed lines are two standard deviations of the noise, ± 0.30 .
- ▶ A curve, a fan or a run of one sign in this plot would mean the model missed something systematic.
- ▶ A single number such as the MSE cannot show that. The plot can.
- ▶ **Read residuals exactly as you would after a regression.**

Source: Session 2 lab notebook, block B.

Extrapolation: Outside the Data It Is a Straight Line



- ▶ Ask the fitted network about $w = 30$, far outside the data. Truth: 9.40. Network: 13.52.
- ▶ A ReLU network is piecewise linear. Beyond the last kink it continues its last segment for ever.
- ▶ A tanh network would flatten instead. Neither knows the law; each extends its own shape.
- ▶ **A network interpolates. Outside the training support its answer is an artefact of the architecture.**

Source: Session 2 lab notebook, block B.

Session 2.4

Next Month's 10-Year Yield

One target, one information set, four models, and a result stated in advance



A Frozen Panel: Series, Download Date, Hash

FRED series	What it is	How it enters
DGS10, DGS3MO	10-year and 3-month Treasury yields, daily	last trading day of each month
FEDFUNDS	effective federal funds rate, monthly	value for month $t - 1$
UNRATE	unemployment rate, monthly	value for month $t - 1$
CPIAUCSL	consumer price index, monthly	12-month inflation, month $t - 1$

- ▶ Downloaded once, on 20 September 2026: 439 monthly rows, the last for August 2026.
- ▶ A **hash** (SHA-256) is a file's fingerprint. The notebook stops if the panel's does not match.
- ▶ **Series, date and hash together say exactly which data produced the numbers.**

Source: Federal Reserve Bank of St. Louis, FRED, fred.stlouisfed.org; Session 2 lab notebook, block C.

The Question



- ▶ Standing at the end of month t , forecast the 10-year Treasury yield at the end of month $t + 1$.
- ▶ Train on 1990 to 2014, choose the design on 2015 to 2019, report on January 2020 to August 2026.
- ▶ The test window holds a pandemic and the fastest tightening in forty years.
- ▶ **Can a network beat “no change”?**

Source: FRED series DGS10, month-end, downloaded 20 Sep 2026.

The Target: Month-End, Not the Monthly Average

- ▶ FRED publishes the 10-year yield daily (DGS10) and as a monthly average of those days (GS10). The target here is the **last trading day of the month**.
- ▶ Averaging a persistent daily series over a month builds serial correlation into its monthly *changes*, even when the daily series is a pure random walk (Working, 1960).
- ▶ A model “forecasting” the change in a monthly average is partly predicting that arithmetic, not the market.
- ▶ Month-end values have no such overlap: the change from t to $t + 1$ uses two points in time.
- ▶ The lab asks you to switch to the monthly average and watch the forecast become easier.
- ▶ **Before comparing models, make sure the target does not manufacture predictability.**

Source: H. Working, “Note on the correlation of first differences of averages in a random chain”, *Econometrica* 28(4), 1960.

The Information Set: What Had Been Published by the End of Month t

Predictor	Dated	Why that date
10-year yield	month-end t	a market price, seen the same day
10-year minus 3-month spread	month-end t	market prices
federal funds rate	month $t - 1$	a monthly average
unemployment rate	month $t - 1$	released early in the next month
12-month CPI inflation	month $t - 1$	released in the middle of the next month

- **Information set:** what the forecaster could know. **Publication lag:** the delay between a month and its statistic.
- These are today's revised figures, the current **vintage**, not the real-time ones.
- **Every predictor is dated $\leq t$. A forecast that uses tomorrow's data is not a forecast.**

Splits by Date; Scalers from the Training Window

train 1990–2014 300 months

weights and scaler statistics come from here only

validate

2015–2019
60 months

test

Jan 2020–Aug 2026
79 months

- ▶ Samples are split by the *date of the label*, in order. No shuffle: next month is not a fresh draw.
- ▶ A **scaler** standardises each input to mean 0 and variance 1. Its mean and standard deviation are estimated on 1990–2014 and then applied, unchanged, to the later windows.
- ▶ Validation data choose the width and the stopping epoch. Test data are touched once.
- ▶ **Anything estimated, including the scaler, is estimated on the training window.**

Four Models, One Information Set

Model	Forecast of y_{t+1}	Estimated
random walk	y_t : no change	nothing
AR(1)	$a + \rho y_t$	2 coefficients
OLS	$a + \beta^\top x_t$, all five predictors	6 coefficients
MLP	$y_t + g_\theta(x_t)$, one hidden ReLU layer	width 4: 29 parameters

- ▶ Every model sees the same five predictors, the same splits, the same scoring. Differences in accuracy are differences in the model, nothing else.
- ▶ The network predicts the *change*. A network that outputs zero is the random walk, so it can only lose by learning something false.
- ▶ **A comparison is fair only when the information sets are identical.**

Said in Advance: the Random Walk Is Hard to Beat

- ▶ A long yield is an asset price. If next month's change were easy to predict, traders would already have moved today's price.
- ▶ The forecasting literature agrees: for interest rates and exchange rates at short horizons, “no change” beats most models out of sample.
- ▶ The test window lies partly outside the training support: 2020 brought unemployment never seen in 1990–2014, and 2022–2023 the fastest tightening in forty years.
- ▶ The curve at $w = 30$ showed what a network does outside its data: it extends its last segment.
- ▶ So the expected result is that the network does *not* win. We say so before looking.
- ▶ **If the network loses to “no change”, that is the lesson of the exercise, not its failure.**

Source: R. Meese and K. Rogoff, “Empirical exchange rate models of the seventies”, *JIE* 14, 1983; G. Duffee, “Forecasting interest rates”, *Handbook of Economic Forecasting* 2A, 2013.

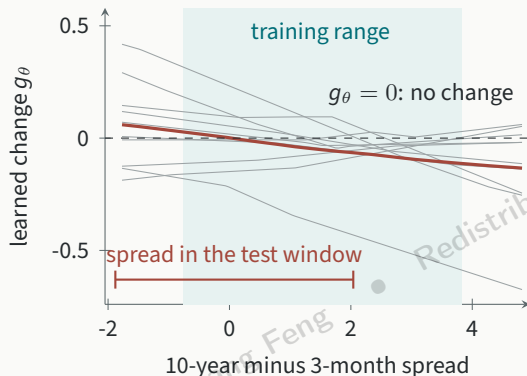
Results, January 2020 to August 2026

Model	RMSE	MAE
random walk	0.2658	0.2097
AR(1)	0.2647	0.2089
OLS, five predictors	0.2855	0.2255
MLP, width 4, mean of 10 seeds	0.3078	0.2406

- ▶ **RMSE**: root mean squared error. **MAE**: mean absolute error. Percentage points, 79 months.
- ▶ The random walk and AR(1) tie. Macro predictors *raise* the error; a network raises it more.
- ▶ Width 4 won on validation: MSE 0.0349, against 0.0469 (width 8) and 0.0403 (16).
- ▶ **More flexibility bought a worse forecast, exactly as announced.**

Source: Session 2 lab notebook, block C; FRED data downloaded 20 Sep 2026.

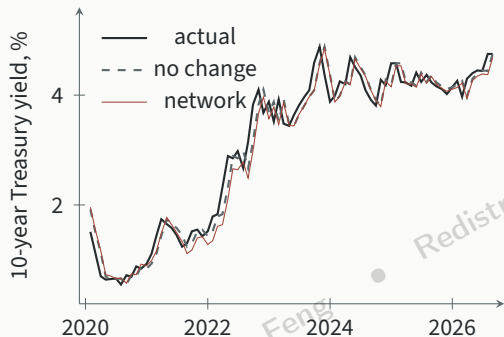
What the Network Actually Learned



- ▶ It predicts the *change*, $\hat{y}_{t+1} = y_t + g_\theta(x_t)$: a network that outputs zero is the random walk.
- ▶ Ten seeds disagree by 0.611 inside the training range and 0.735 at the edges, against a whole error of RMSE 0.2658.
- ▶ Outside that range the ReLU segments simply continue straight, as at $w = 30$.
- ▶ **The seeds disagree by as much as the quantity being forecast.**

Source: Session 2 lab notebook, block C: the width-4 network's learned change, other predictors at their training means.

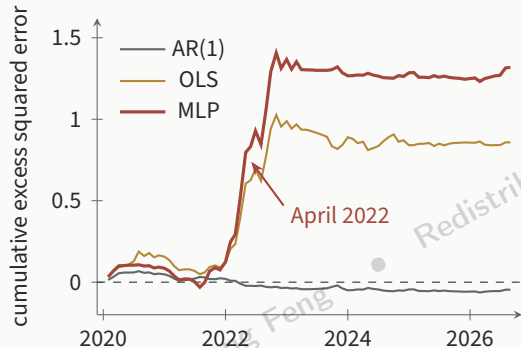
Actual and Forecast, Month by Month



- ▶ Actual, “no change” and the ten-seed mean over the 79 test months: three lines, one path.
- ▶ Every forecast is essentially last month’s level. The level is persistent; the change is not.
- ▶ The gaps in the table are hundredths of a point, invisible at this scale.
- ▶ **That the three lines are indistinguishable is itself the result.**

Source: Session 2 lab notebook, block C; FRED series DGS10, month-end, downloaded 20 Sep 2026.

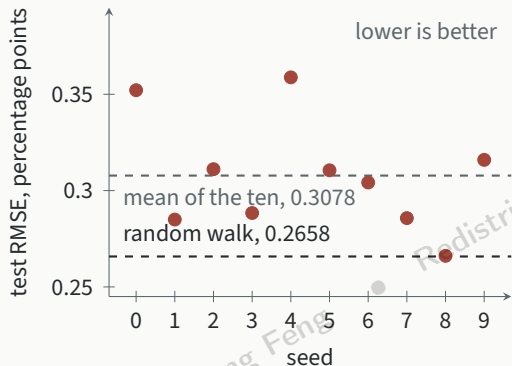
When Did the Network Lose?



Source: Session 2 lab notebook, block C.

- ▶ Each month's squared error minus the random walk's, added up: above zero the model is losing.
- ▶ At the end of the window: AR(1) -0.046 , OLS $+0.858$, MLP $+1.319$.
- ▶ The worst month alone is April 2022, excess 0.281 : 21% of the network's whole loss.
- ▶ **One RMSE over 79 months cannot say when a model lost. This can.**

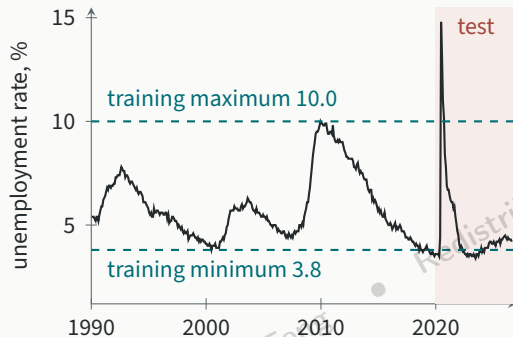
Ten Seeds: a Network Is a Distribution of Answers



- ▶ Only the initial weights differ, but the loss is not convex: the draw selects *which* stationary point you reach. Ten functions, not noise.
- ▶ The seed range is 0.0927; the whole gap from random walk to network is 0.0297, $3.1\times$ smaller.
- ▶ The **ensemble** — average the ten forecasts, then score — gives 0.2956 and still loses.
- ▶ **One seed could have “shown” a win. Report the range and the ensemble.**

Source: Session 2 lab notebook, block C: ten seeds of one width-4 network, 29 parameters, 300 monthly observations.

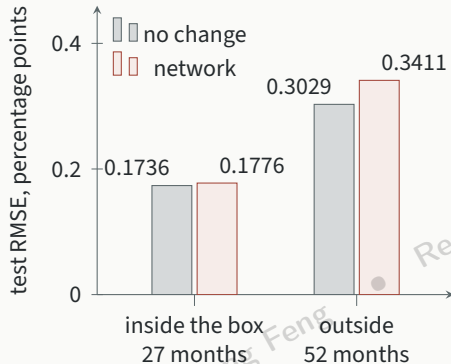
Outside the Training Support



- ▶ Unemployment is outside its training range in 37% of test months: above 10.0 in 2020, below 3.8 later.
- ▶ Likewise the spread (24%), the yield (22%), inflation (18%) and the funds rate (4%).
- ▶ Jointly it is worse: 66% of months are outside on some predictor; there the network extrapolates, as at $w = 30$.
- ▶ **Check the support before believing a flexible model; the next slide splits the errors by it.**

Source: Session 2 lab notebook, block C; FRED series UNRATE, downloaded 20 Sep 2026.

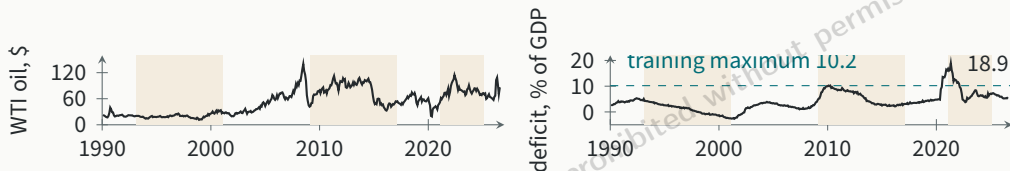
It Loses Where It Extrapolates



- Split the 79 test months by whether every predictor lies inside its 1990–2014 range.
- Inside, the network’s excess over “no change” is $+0.0040$; outside, $+0.0382$, which is $9.5\times$ larger.
- A month inside every marginal range can still be outside the joint one: 66% are, against 37% for unemployment.
- **The network does not lose everywhere. It loses where it has never been.**

Source: Session 2 lab notebook, block C.

More Information, Measured



- ▶ Four added, all dated $\leq t$: **oil** (month-end WTI, a market price, dated t), the federal **deficit** (12-month rolling total, % of GDP, dated $t - 1$), and two party dummies.
- ▶ The rolling sum removes the very large April tax seasonality, at the cost of a year of history plus its lag: 438 rows, 299 training months against the 300 above.
- ▶ `pres_dem` (shaded) switches 6 times in 438 months, twice in the 79 test months; `unified` 13 times, 3 in the test window: one switch per 73 months.
- ▶ **Six changes in thirty-six years is six observations, not 438: a dummy constant across a window is a level shift to memorise, not a predictor.**

Source: Federal Reserve Bank of St. Louis, FRED, downloaded 21 September 2026; party control from the public record.

The Result, with Nine Predictors

Model	RMSE	MAE
random walk	0.2658	0.2097
AR(1)	0.2646	0.2089
OLS, five predictors	0.2842	0.2244
OLS, nine predictors	0.3163	0.2500
MLP, five predictors, ensemble of ten seeds	0.2929	0.2271
MLP, nine predictors, ensemble of ten seeds	0.3021	0.2374

- ▶ Same splits and scoring, four more predictors: OLS got 11% worse, the network 3%.
- ▶ Validation chose width 4 with five predictors (MSE 0.0349) and width 16 with nine (0.0403); no width did better, and the share of test months outside the training box rose from 66% to 72%, the deficit outside in 24% of them and oil in none.
- ▶ **More information widened where the model extrapolates, and cost more than it was worth.**

Source: Federal Reserve Bank of St. Louis, FRED, downloaded 21 September 2026; same target, splits and scoring as above.

What Session 5 Inherits: a Frozen, Dated Baseline

- ▶ Two files leave this deck: the frozen panel, and the four models' test-window forecasts, each row stamped with the date the forecast was made and the date of its target.
- ▶ Session 5 turns central-bank text into a numerical signal. A signal dated $\leq t$ is merged on the forecast date and must improve on *these* numbers.
- ▶ The baseline does not move: the frozen panel, its hashes and the four models' forecasts are untouched, and the nine-predictor run is a second experiment beside it, not a replacement. Whatever text adds is measured against RMSE 0.2658, under the same splits and information rule.
- ▶ The bar is high on purpose: “no change” is what a new data source has to beat.
- ▶ **A new signal is worth what it adds to a fixed, honest baseline, and nothing more.**

Source: Session 2 lab notebook, block C.

Session 2.5

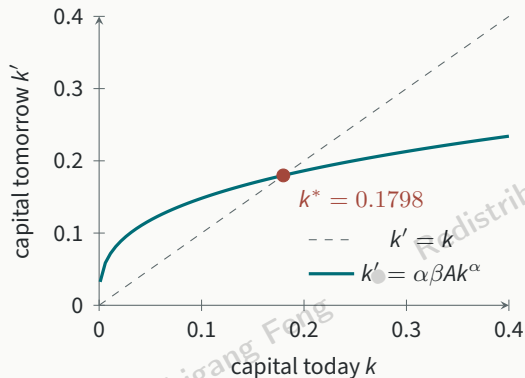
Solving the Growth Model as Supervised Learning on the Euler Equation

The network is the policy, sampled states are the inputs, and the Euler equation supplies the labels



Zhigang Feng

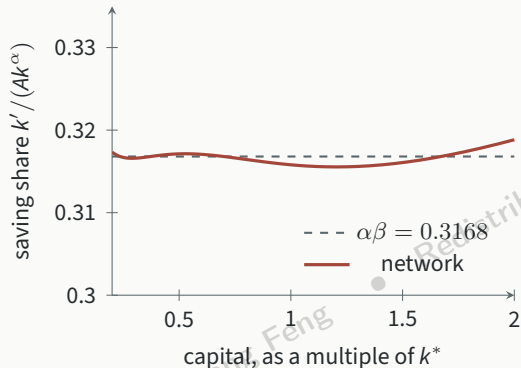
Session 1 Solved This in One Second. Why?



- ▶ The growth model of Session 1.4: choose k' to maximise $\sum_t \beta^t u(c_t)$ subject to $c + k' = Ak^\alpha + (1 - \delta)k$.
- ▶ With log utility and $\delta = 1$ the **policy function** is known: $k' = \alpha\beta Ak^\alpha$.
- ▶ In the Session 1 lab a network recovered it to within 0.44% in under a second.
- ▶ **Fast and accurate. Before trusting the method, ask what it actually had to learn.**

Source: Calibration $\alpha = 0.33, \beta = 0.96, A = 1, \delta = 1$, log utility. Session 1 lab notebook; Brock & Mirman, *JET* 4(3), 1972.

The Reveal: the Target Was One Number



- ▶ Write the policy as a share of output: $k'/(Ak^\alpha) = \alpha\beta$, the same at every k .
- ▶ The network's learned share lies between 0.3155 and 0.3188 everywhere. It learned a constant.
- ▶ One bias term could pass this test. It says the code is right; it says little about approximation.
- ▶ **So this deck climbs a ladder: the same code, on problems where the share is a real function.**

Source: Session 2 lab notebook, block D (rung 0: log utility, $\delta = 1$, $\alpha = 0.33$, $\beta = 0.96$, $A = 1$).

Session 2.5

The Residual as a Loss

Where the labels come from when there are no data



Solving a Model as Supervised Learning: the Dictionary

Supervised learning	Digits (Decks 2.1–2.3)	Growth model (this deck)
inputs x_i	images	sampled states k_i (and z_i)
model f_θ	classifier	the policy $k' = \pi_\theta(k, z)$
prediction error	$\hat{p}$ against the label	the Euler residual at (k_i, z_i)
label	the true digit	zero , at every state

- ▶ Nobody hands us the right k' at each k . The model's own optimality condition says what a right answer must satisfy.
- ▶ Inputs are free: we draw as many states as we like, fresh at every step.
- ▶ **The Euler equation supplies the labels. Everything in Decks 2.3 and 2.4 then applies unchanged.**

The Euler Equation, and Its Unit-Free Residual

- ▶ The **Euler equation**: at an optimum, giving up one unit of consumption today and eating the proceeds tomorrow must leave utility unchanged,

$$u'(c) = \beta E[u'(c') R'], \quad R' = \alpha z' A k'^{\alpha-1} + 1 - \delta.$$

- ▶ Under a candidate policy π_θ both sides can be computed at any state. Their gap is the **residual**.
- ▶ A gap in marginal-utility units means nothing by itself. Make it **unit-free**: divide, and take logs,

$$r_\theta(k, z) = \log \frac{\beta E[u'(c') R']}{u'(c)}.$$

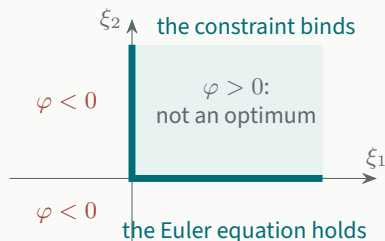
- ▶ $r = 0.01$ reads as a one per cent wedge between the two sides, at any calibration.
- ▶ **The lab code uses exactly this log residual, on all three rungs.**

Necessary, Not Sufficient

- ▶ The Euler equation is a *first-order* condition. The true policy satisfies it. So do other paths.
- ▶ The full solution also needs the **transversality condition**: capital must not be accumulated, or run down, without bound. The Euler equation alone does not impose it.
- ▶ A residual is also *local*: it compares today with tomorrow at one state. Small errors at every state can still compound along a simulated path.
- ▶ A loss built on the Euler equation therefore cannot certify the policy that minimises it.
- ▶ **A small residual is evidence. It is not a proof. Rung 1, later in this deck, measures the difference.**

Source: N. Stokey, R. Lucas, E. Prescott, *Recursive Methods in Economic Dynamics*, 1989, §4.5.

When the Euler Equation Holds as an Inequality

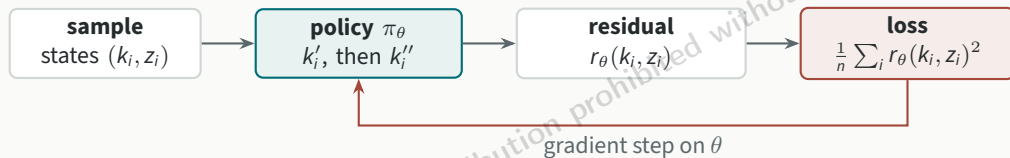


The thick corner is $\varphi = 0$: all three conditions at once. Architecture is exact where the shape can be written in, as the sigmoid saving share is; this residual is general; a penalty is approximate.

Source: A. Fischer, “A special Newton-type optimization method”, *Optimization* 24, 1992; Maliar, Maliar & Winant, *JME* 122, 2021.

- ▶ With $k' \geq \underline{k}$, optimality is a **Kuhn–Tucker** (KKT) system: slack $\xi_1 = k' - \underline{k} \geq 0$, Euler wedge $\xi_2 \geq 0$, $\xi_1 \xi_2 = 0$ — **complementary slackness**, an **occasionally binding** constraint.
- ▶ A squared residual of an equality cannot express an inequality. $\min(\xi_1, \xi_2) = 0$ can, but it is kinked: zero gradient on the branch it drops, Deck 2.3’s dying ReLU in the loss.
- ▶ **Fischer–Burmeister**, $\varphi = \xi_1 + \xi_2 - \sqrt{\xi_1^2 + \xi_2^2}$, is zero exactly on the corner, and not differentiable there. **Session 4 builds it and runs it.**

From Residual to Loss: Zero Is the Label



- ▶ Squared error against a label of zero: $\ell_i = (r_\theta(k_i, z_i) - 0)^2$. The training loop is Deck 2.4's five lines.
- ▶ The policy is evaluated twice, at k and at k' , so gradients flow through both.
- ▶ **No benchmark policy ever enters the loss. Benchmarks are kept for testing.**

The Weighted-Residual Reading

- ▶ Classical **projection methods** write the policy as $\sum_j a_j \phi_j(k)$ with fixed basis functions (Chebyshev polynomials) and choose a to make the residual small.
- ▶ They differ in what “small” means: *collocation* sets $r = 0$ at chosen nodes; *least squares* minimises $\int r^2$; *Galerkin* makes r orthogonal to the basis.
- ▶ The network method is the least-squares member of this family, with two changes: the basis is learned, and the integral is a Monte Carlo average over sampled states.
- ▶ The sampling distribution is the weight function. Where you sample is where you ask for accuracy.
- ▶ **This is Judd’s projection method with a neural basis. The economics has not changed.**

Source: K. Judd, “Projection methods for solving aggregate growth models”, *JET* 58(2), 1992; *Numerical Methods in Economics*, 1998, ch. 11.

What Is Different from Regression on Labels

	Regression on labels	Regression on an equation
data	fixed, finite, noisy	unlimited, drawn fresh, noiseless
overfitting	to the sample's noise	none to noise; errors hide where no state was drawn
the model appears	once, $f_{\theta}(x_i)$	twice, $\pi_{\theta}(k)$ and $\pi_{\theta}(\pi_{\theta}(k))$
a zero loss means	a perfect fit	the first-order condition holds on the sample

- ▶ Against collocation: no grid, no fixed basis, and no linear system. Cost grows with the sample, not with N^d .
- ▶ **Held-out testing survives the change. Its job moves from noise to coverage.**

Three Rungs

Rung	Model	What it tests	Benchmark
0	log utility, $\delta = 1, \beta = 0.96$	the code	$k' = \alpha\beta Ak^\alpha$
1	CRRA $\gamma = 2, \delta = 0.025, \beta = 0.99$	a share that varies	time iteration
2	rung 0 with AR(1) productivity z	the expectation	$k' = \alpha\beta zAk^\alpha$

- ▶ All rungs: $\alpha = 0.33, A = 1$. **CRRA**: $u(c) = c^{1-\gamma}/(1-\gamma)$. Rung 2: $\log z' = 0.9 \log z + 0.05 \varepsilon'$.
- ▶ **Steady state** k^* , where $k' = k$: 0.1798 on rungs 0 and 2, 28.35 on rung 1.
- ▶ **One code path, one new difficulty per rung, and an answer we never trained on.**

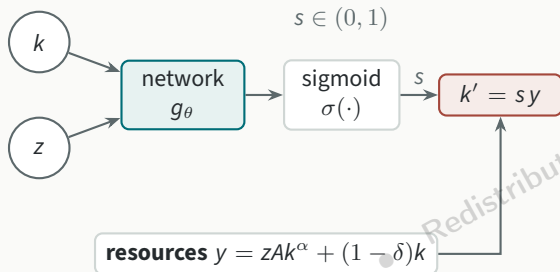
Session 2.5

Feasibility and Expectations

What to build into the network so it cannot cheat, and how to compute $E[\cdot]$



Feasibility by Construction: the Sigmoid Share



- **Feasibility:** $c > 0$ and $k' > 0$. One infeasible state makes $\log c$ undefined and ends training.
- The network outputs a *saving share* $s = \sigma(g_\theta) \in (0, 1)$, so $k' = sy$ and $c = (1 - s)y$ are positive at every θ .
- Inputs are $\log k$ and $\log z$, rescaled to $[-1, 1]$.
- **A constraint written into the architecture cannot be violated, and costs no penalty term.**

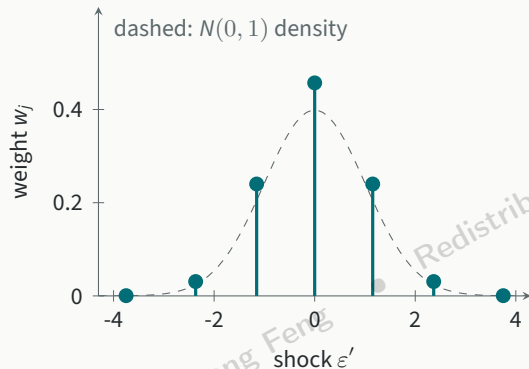
The Stochastic Case: an Expectation Inside the Residual

- ▶ Rung 2: productivity follows $\log z' = \rho \log z + \sigma \varepsilon'$, $\varepsilon' \sim N(0, 1)$, with $\rho = 0.9$, $\sigma = 0.05$. The state is (k, z) .
- ▶ The residual now contains an integral over tomorrow's shock:

$$r_{\theta}(k, z) = \log \frac{\beta E_{\varepsilon'} [u'(c'(\varepsilon')) R'(\varepsilon')]}{u'(c)}.$$

- ▶ Two expectations are in play. The *outer* one, over states, is the loss, and sampling it is harmless.
- ▶ The *inner* one, over ε' , sits inside a logarithm and a square. Sampling it is not harmless.
- ▶ **How the inner expectation is computed decides whether the loss is the right loss.**

Gauss-Hermite, and Why the Loss Stays Differentiable



► Gauss-Hermite quadrature:

$E[h(\varepsilon)] \approx \sum_{j=1}^J w_j h(\varepsilon_j)$, with nodes and weights fixed in advance. Exact for polynomials up to degree $2J - 1$.

- $J = 7$ here: seven evaluations of the policy per state, stacked in one batch.
- A fixed weighted sum is an ordinary differentiable function of θ : autograd passes straight through.
- **No sampling noise enters the inner expectation.**

Source: Judd, *Numerical Methods in Economics*, 1998, §7.2. Nodes and weights: Session 2 lab notebook, block D.

Why the Square of a Monte Carlo Mean Is Biased

Inner expectation by	average of (estimate) ²	Var(X)/ n
Monte Carlo, $n = 1$ draw	2.5×10^{-3}	2.5×10^{-3}
Monte Carlo, $n = 10$ draws	2.5×10^{-4}	2.5×10^{-4}
Monte Carlo, $n = 100$ draws	2.5×10^{-5}	2.5×10^{-5}
Gauss–Hermite, 7 nodes	4×10^{-16}	none

- ▶ For a mean $\bar{X}$ of n draws, $E[\bar{X}^2] = (EX)^2 + \text{Var}(X)/n$. The test function here has $EX = 0$.
- ▶ A squared residual with a sampled inner mean is minimised at the wrong policy.
- ▶ **Quadrature removes the bias; the alternative is two independent draws multiplied together.**

Source: Session 2 lab notebook, block D; test function $e^{0.05\varepsilon} - e^{0.05^2/2}$. Maliar, Maliar & Winant, *JME* 122, 2021.

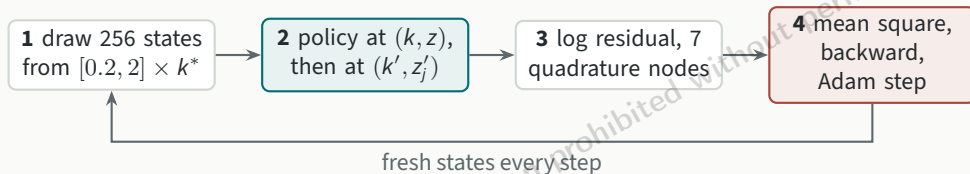
Session 2.5

Algorithm and Validation

Train on residuals; believe only what was tested elsewhere



The Algorithm



- Network: two tanh hidden layers of 32 units (1,153 parameters with one input). Learning rate on a cosine decay.
- Steps: 500 on rung 0, 3,000 on rung 1, 1,500 on rung 2. Seconds on one CPU core: 1.4, 4.6, 4.0.
- **Then stop training and start testing: held-out states, off-domain states, benchmark, steady state.**

Source: Session 2 lab notebook, block D. Azinovic, Gaegauf & Scheidegger, “Deep equilibrium nets”, *IER* 63(4), 2022.

The Code: Policy and Residual

```
def saving_share(self, k, z):
    x = (torch.log(k) - self.log_center) / self.log_scale
    if self.rung.stochastic:
        x = torch.cat([x, torch.log(z) / self.rung.log_z_bound], dim=1)
    return torch.sigmoid(self.net(x))

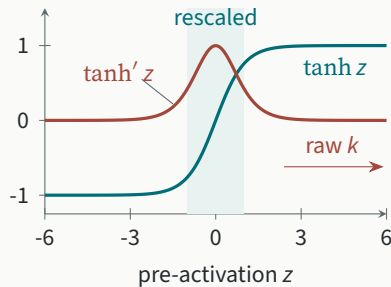
def forward(self, k, z):
    return self.resources(k, z) * self.saving_share(k, z)

def log_residual(model, k, z):
    c, rhs = expected_rhs(model, k, z)
    return torch.log(rhs) + model.rung.gamma * torch.log(c)

loss = log_residual(model, k, z).square().mean()
```

- `expected_rhs` returns c and $\beta E[u'(c')R']$. **Feasibility is line 8; the economics is line 12.**

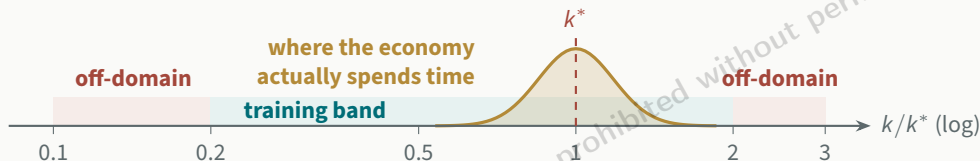
Why the Inputs Are Rescaled



- ▶ Rung 1 runs over $k \in [5, 57]$. A one-input layer starts with weights near $U(-1, 1)$, so raw k makes the pre-activations tens.
- ▶ $\tanh'(10) = 8 \times 10^{-9}$: the first layer saturates and nothing trains — Deck 2.2's vanishing gradient, from input scale, not depth.
- ▶ Mixed scales also stretch the loss contours (Deck 2.3), and Xavier and He assume unit-variance inputs.
- ▶ **The sigmoid share is the same idea at the other end: a bounded, order-one target.**

Source: Glorot & Bengio, *AISTATS* 9, 2010; He et al., *ICCV*, 2015. Session 2 lab notebook, block D.

Where to Sample



- ▶ Training states are uniform in $\log k$ on $[0.2, 2] \times k^*$, and within three standard deviations of $\log z$. Rung 2 itself keeps k inside $[0.72, 1.38] \times k^*$ 95% of the time.
- ▶ **Held-out** states: fresh draws from the same band, from a generator training never used; they test coverage. **Off-domain** states: $[0.1, 0.2) \cup (2, 3] \times k^*$, never asked of the network; they test extrapolation.
- ▶ **The sampling band is a modelling choice: the solution is only claimed where it was trained and tested.**

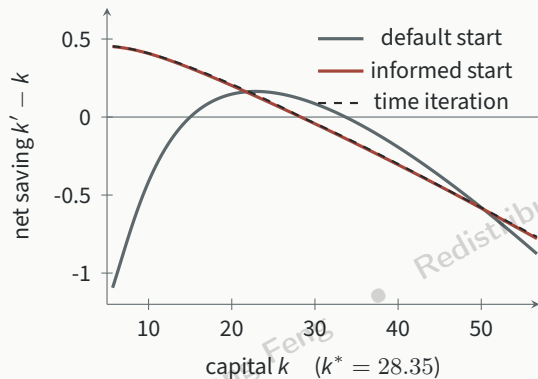
Source: Ergodic density: the stationary distribution of $\log k$ under $k' = \alpha\beta z A k^\alpha$, standard deviation 0.165.

Why Sample at All

- ▶ A grid costs N^d points; drawing n states costs n whatever d is. Sampling is how the curse of dimensionality is dodged operationally, and it is the whole case for the method.
- ▶ Sampling also fixes what “small residual” means. The loss $\min_{\theta} E_{\mu}[r_{\theta}^2]$ is a weighted L^2 norm, and the sampling density μ is the weight.
- ▶ Uniform μ asks for uniform accuracy across the band; the **ergodic distribution** asks for accuracy where the economy spends its time. Different problems, different answers.
- ▶ Neither is right in general. The choice says which states the solution is for, so state it, and report the error apart where you did not sample.
- ▶ **Forward: once the policy generates its own training states, μ moves with θ and stops being a free choice — Sessions 3 and 4.**

Source: K. Judd, *Numerical Methods in Economics*, 1998, ch. 11; Judd, Maliar & Maliar, *Quantitative Economics* 2(2), 2011.

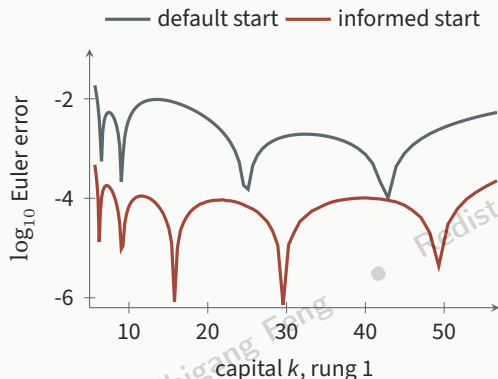
Rung 1, Twice: a Small Residual with the Wrong Policy



- ▶ Default start: $\log_{10}$ max Euler error -1.74 , policy wrong by 25%, and simulated capital falls to 0.06, not 28.35.
- ▶ Informed start (share set to $k^*/y(k^*)$ at initialisation): policy error 0.04%, simulated $k = 28.30$.
- ▶ With $\beta R \approx 1$ each quarter, the residual is nearly flat in the policy.
- ▶ **The loss did not catch this. The benchmark and the steady state did.**

Source: Session 2 lab notebook, block D (rung 1: CRRA $\gamma = 2$, $\delta = 0.025$, $\alpha = 0.33$, $\beta = 0.99$; same seed, same 3,000 steps).

Held-Out and Off-Domain Residuals, on Judd's Scale

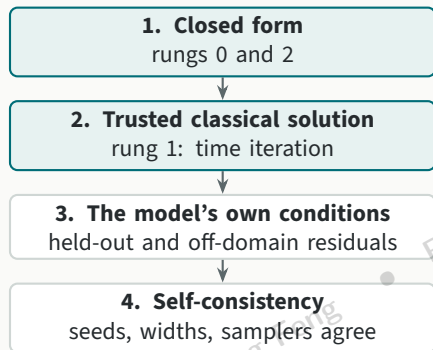


$\log_{10}$ max error	rung 0	rung 1	rung 2
held-out	-1.95	-3.34	-2.15
off-domain	-1.29	-1.59	-1.42

- **Judd's scale:** $\log_{10}$ of the consumption error implied by the residual. -3 is one dollar in a thousand.
- Off-domain, accuracy falls by half a decade to almost two.
- **Report the maximum, and report it where the network was not trained.**

Source: Session 2 lab notebook, block D; calibrations as on "Three Rungs". Judd, *JET* 58(2), 1992.

The Benchmark Hierarchy



- ▶ Validate against the strongest benchmark available, and say which one you had.
- ▶ **Time iteration**: guess tomorrow's policy, solve the Euler equation for today's k' at each grid point, repeat until the policy stops moving. It iterates on the policy itself.
- ▶ **Climb down, never up: prove the code where level 1 exists, then carry the same code further.**

Source: Miao & Feng, *AI for Economists*, ch. 10; W. Coleman, "Equilibrium in a production economy with an income tax", *Econometrica* 59(4), 1991.

The Steady-State Check, and the Gates

Gate	rung 0	rung 1	rung 2
feasible at every tested state	yes	yes	yes
max policy error, held-out (gate: below 1%)	0.64%	0.04%	0.56%
max policy error, off-domain (reported)	3.4%	1.3%	2.9%
simulated against analytical k^* (gate: 1%)	0.47%	0.18%	0.00%

- ▶ Steady-state check: from $0.5 k^*$, apply the learned policy 3,000 times with $z = 1$. A *global* test: local errors that compound show up here.
- ▶ **A gate is fixed before the run. All rungs pass; rung 1 from the default start fails two of three.**

Source: Session 2 lab notebook, block D; calibrations as on “Three Rungs”; policy errors against the closed form (rungs 0, 2) and time iteration (rung 1).

Time Iteration and the Network, Side by Side

Rung 1	time iteration	network
numbers stored	400 grid values	1,153 parameters
seconds, one CPU core	1.5 (292 iterations)	4.6 (3,000 steps)
$\log_{10}$ max Euler error, held-out band	-3.44	-3.34

- ▶ In one dimension the classical method ties on accuracy and wins on the rest. A five-method table from the instructor's quantitative-macro course (another calibration) agrees.
- ▶ A grid costs N^d ; the network's cost grows with the sample. Its case is dimension.
- ▶ **Learn the method where it loses, so that you can trust it where nothing else runs.**

Source: Session 2 lab notebook, block D (rung 1: CRRA $\gamma = 2$, $\delta = 0.025$, $\alpha = 0.33$, $\beta = 0.99$).

What This Deck Did Not Do — and Where Each Idea Goes Next

Built in Session 2

Used next

held-out discipline

every later session

an equation's residual as the loss

Session 4: heterogeneous-agent equilibrium conditions

the network as a *policy*

Session 3: the network as a *value*, and learning by acting

a frozen, dated yield baseline ●

Session 5: does a text signal improve on it?

gates fixed before the run

Session 6: verifying work delegated to agents

- ▶ No value function was approximated today: network and benchmark are both policies.
- ▶ **Next, Deck 2.6: the policy was a plain dense network. What if the architecture itself carried the economics?**

Session 2.6

Architecture as a Constraint on W , and What More Parameters Buy

Restrictions that encode what we know about the input, and the opposite move: more parameters than observations



Zhigang Feng

The Same Layer, with Restrictions

- ▶ Every network so far has been **dense**: every entry of every W free, nothing assumed about the input.
- ▶ A dense layer is indifferent to its input's structure. Shuffle the pixels of every digit the same way and it learns exactly as well — which means it never used the fact that pixels have neighbours.
- ▶ **An architecture is a constraint on W** : entries fixed at zero, entries forced equal, entries forced non-negative. Each one is a claim about the target.
- ▶ Three of those claims are worth an economist's time: locality (convolutions), shape (monotone and convex policies), and order — or its absence (sequences, networks of agents).
- ▶ **Then the opposite question: what does an *unrestricted* network buy from having more parameters than observations?**

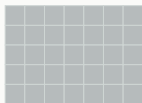
Session 2.6

Architecture as a Constraint on W

The same layer, with restrictions that encode what we know about the input

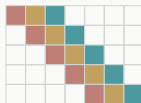


One Idea, Three Times



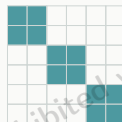
dense

every entry free



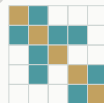
convolution

sparse and repeated



recurrent

one block at
every date



graph

follows the links

- ▶ A layer is $\sigma(Wx + b)$. White cells are fixed at zero; in the dense matrix every grey cell is free; in the other three, cells of one colour are *one* shared parameter.
- ▶ A dense W assumes nothing about the input, so everything must be learned from data.
- ▶ **A constraint on W is a prior about the target. True, it saves parameters and data. False, it is bias.**

Source: Miao & Feng, *AI for Economists*, §9.6.

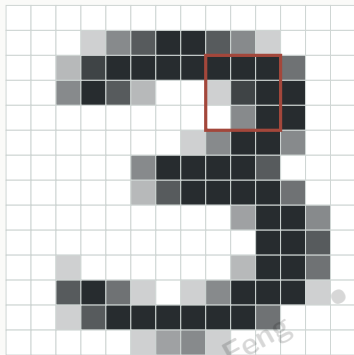
Session 2.6

Locality: the Convolution

Two constraints on W — sparse, and repeated

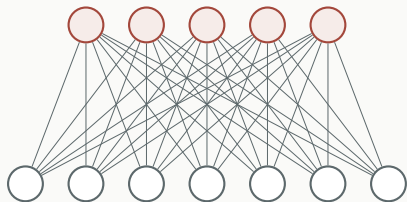


Back to the Digit: What a Dense Layer Ignores

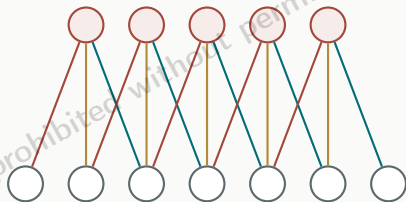


- ▶ Deck 2.1 flattened the image into a vector. A dense first layer then has one weight per pixel per hidden unit: $196 \times 128 = 25,088$ weights for the 14×14 digit here, $784 \times 128 = 100,352$ for MNIST's 28×28 .
- ▶ Shuffle the pixels, the same way in every image, and a dense network learns exactly as well. It never knew which pixels were neighbours.
- ▶ Yet a stroke is a local object: the red 3×3 patch means the same thing wherever it sits.
- ▶ **Two facts about images are unused: nearby pixels belong together, and a pattern can appear anywhere.**

Dense vs Convolution: 35 Weights vs 3



dense: 35 lines, 35 free weights

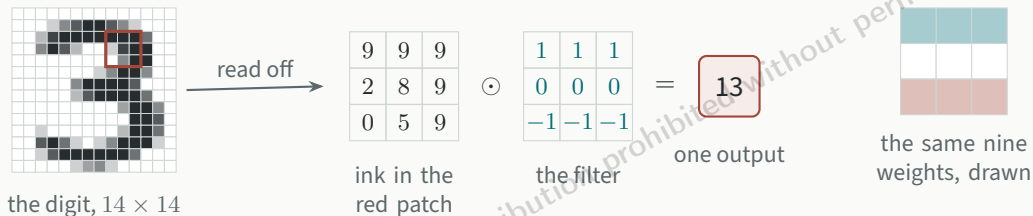


convolution: 15 lines, 3 free weights

- ▶ Seven inputs, five outputs. On the right each output sees three adjacent inputs (**local connection**), and lines of one colour are one parameter (**weight sharing**).
- ▶ The three shared weights are the **filter**, or **kernel**. Sliding it along the input is a **convolution**.
- ▶ **Convolution is a dense layer after two constraints on W : sparse, and repeated.**

Source: Miao & Feng, *AI for Economists*, §9.6, Fig. 9.9.

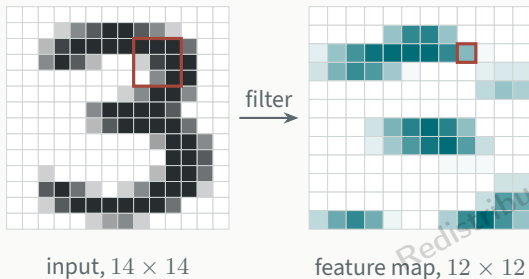
The Receptive Field



- ▶ The **receptive field** of an output is the part of the input it can see: here the 3×3 patch outlined in red, and nothing else.
- ▶ The output is an inner product: $(9 + 9 + 9) - (0 + 5 + 9) = 13$. The filter asks whether there is ink above and paper below, so it fires on the lower edge of a horizontal stroke.
- ▶ **Drawn as a picture it is the template neuron of Deck 2.1, shrunk to nine weights — and those nine are reused at every position on the next slide.**

Source: Patch: rows 3 to 5, columns 9 to 11 of the 14×14 digit of Deck 2.1.

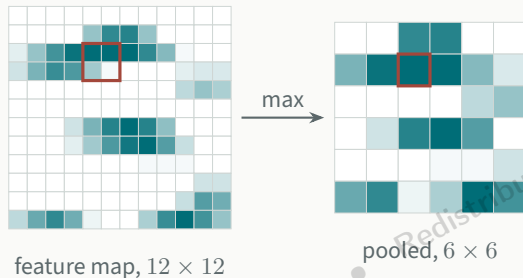
Feature Maps: One Question Asked Everywhere



- ▶ Slide the filter over every position, apply a ReLU, and record the answers: a **feature map**.
- ▶ It lights up along the lower edges of horizontal strokes, wherever they are.
- ▶ A layer has many filters, often 32 or 64, each with its own map. They are learned, not designed.
- ▶ **One filter is one question, asked at every location with the same nine weights.**

Source: Computed from the digit of Deck 2.1 with the filter of the previous frame; the red cell is the value 13.

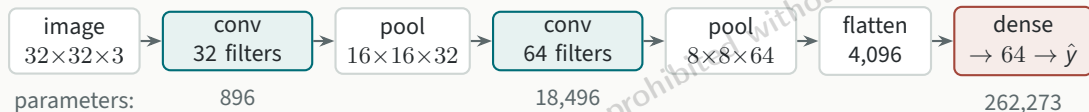
Pooling



- **Pooling** replaces each small block by one summary number. **Max pooling** keeps the largest.
- The red 2×2 block holds 27, 27, 10, 1; it becomes 27. Resolution halves: $12 \times 12 \rightarrow 6 \times 6$.
- The question becomes “is the edge somewhere in this block?” Pooling has no parameters.
- **Convolution extracts features; pooling reduces resolution, trading location for robustness.**

The Full Pipeline: Equivariance, then Approximate Invariance

A colour image now, not the digit: three channels, 32×32 pixels.



- **Equivariance**: shift the input one pixel and the feature map shifts one pixel (weight sharing).
- **Invariance**: the output does not change. Pooling gives it approximately, for small shifts.
- Of 281,665 parameters the convolutions hold 6.9%; the dense head holds 93.1%.
- **A convolutional network is a learned feature extractor followed by the network you already know.**

Source: Miao & Feng, *AI for Economists*, §9.6, Fig. 9.10.

What the Constraint Buys, and What It Assumes

- ▶ A 256×256 colour image is 196,608 numbers: a dense layer into 256 units needs about 5×10^7 weights, while sixty-four 3×3 filters over three colour channels need $64 \times (3 \cdot 3 \cdot 3 + 1) = 1,792$ — four orders of magnitude fewer.
- ▶ Fewer parameters need fewer observations. In economics the sample is what it is, so this is decisive.
- ▶ The assumption bought: nearby inputs are related, distant ones are not, and a pattern means the same thing everywhere.
- ▶ Apply a convolution to data with no spatial order, such as the columns of a survey, and the constraint is simply wrong.
- ▶ Random, untrained filters with a ridge regression on top match deep training on satellite tasks (Rolf et al., 2021): the locality and the sharing do the work.
- ▶ **Parameters saved are paid for with an assumption. Check the assumption, not the parameter count.**

Source: E. Rolf et al., *Nature Communications* 12, 2021; Miao & Feng, *AI for Economists*, §9.6.

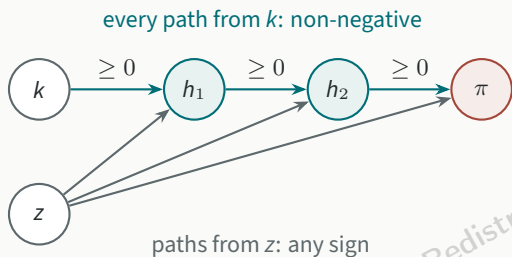
Session 2.6

Shape You Can Guarantee

Monotone and convex by construction, and what the guarantee costs



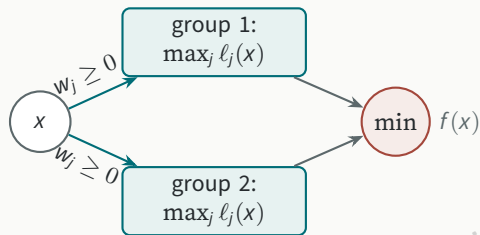
Monotone Policies: Which Weights Carry the Sign



- ▶ Theory often gives the shape: saving rises in wealth, demand falls in own price.
- ▶ A **monotone network**: every weight on every path from x_j non-negative and activations non-decreasing give $\partial f / \partial x_j \geq 0$ everywhere.
- ▶ *Partial* monotonicity is the useful case: constrain the paths from k , leave those from z .
- ▶ **Impose $W \geq 0$ by training V in $W = \text{softplus}(V)$; a clip stalls the gradient at the boundary.**

Source: J. Sill, “Monotonic networks”, *NeurIPS* 10, 1998.

What the Sign Constraint Costs, and What It Does Not

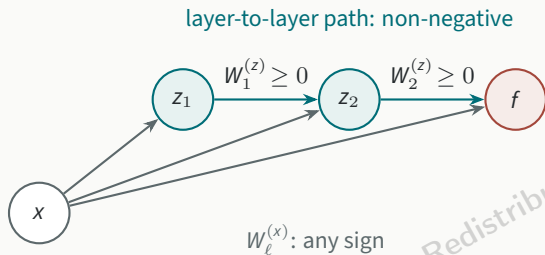


$\ell_j(x) = w_j x + b_j$:
Sill's min-max network

- ▶ Sign constraints shrink the parameter set, so a constrained network often trains more slowly and to a *higher* training loss.
- ▶ That is a harder search, not a smaller class: maxima within groups of linear units, a minimum across groups, and the family is universal among monotone functions.
- ▶ Penalising violations instead stays flexible, and holds only where it was sampled.
- ▶ **The unconstrained network reaches its lower loss partly with functions the theory forbids.**

Source: J. Sill, "Monotonic networks", *NeurIPS* 10, 1998, §2 (the min-max architecture).

Input-Convex Networks: Which Weights, Which Arguments, Which Direction



- ▶ An **input-convex network**:
 $z_{\ell+1} = \sigma(W_\ell^{(z)} z_\ell + W_\ell^{(x)} x + b_\ell)$.
- ▶ Convex in x if $W_\ell^{(z)} \geq 0$ and σ is convex and non-decreasing (ReLU, softplus). The direct weights $W_\ell^{(x)}$ are free.
- ▶ Concave: take $-f$. Convex in some arguments only: constrain only their paths.
- ▶ **Examples: a cost function convex in input prices; a production function concave in capital.**

Source: B. Amos, L. Xu, J. Z. Kolter, “Input convex neural networks”, *ICML*, 2017.

Measured: What an Unconstrained Network Does to Convexity

	median RMSE	worst negative curvature
plain ReLU network	0.00172	5.93 (range 0 to 10.4 over seeds)
input-convex network	0.01345	below 7.1×10^{-11}

- ▶ Target: $0.6x^2 + 0.25x + 0.1$, exactly convex, curvature 1.2 everywhere. 1,153 parameters, five seeds.
- ▶ The plain network fits the level eight times better and is locally concave by about 5.9. Its loss shows none of that.
- ▶ At one seed in five it came out convex anyway: you cannot tell without checking.
- ▶ **The constrained network buys a guarantee and pays for it in fit: 7.8 times the RMSE.**

Source: Recorded in the instructor's quantitative-macro course computations (2026); not recomputed in this session's lab.

What Structure Buys, and What It Refuses

- ▶ **Buys reliability.** The shape holds at every θ , every seed, and outside the training band, where data cannot enforce it.
- ▶ **Buys well-behaved downstream problems.** Convex costs give monotone factor demands, so a market-clearing price is unique and a calibration loop converges.
- ▶ **Buys data efficiency.** Fewer functions to choose between means fewer states to pin one down.
- ▶ **Refuses flexibility.** If the true function violates the shape, the constraint is bias. It must come from theory, not convenience.
- ▶ Session 3 uses the same device for value functions.
- ▶ **Deck 2.5 imposed exactly one of these, feasibility, because it is the one theory guarantees on every rung.**

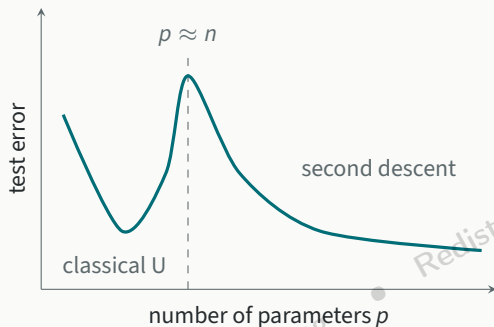
Session 2.6

What More Parameters Buy

The opposite move: no constraint at all, and more parameters than observations



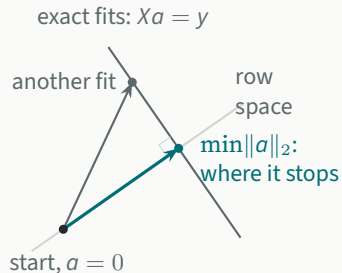
The Modern Twist: Double Descent



- ▶ Test error follows the textbook U, then peaks at the **interpolation threshold** $p \approx n$, where the model first fits the sample exactly.
- ▶ With $p \gg n$ it falls again: **double descent** (a schematic).
- ▶ In forecasting, $p > n$ can lower out-of-sample risk.
- ▶ **More parameters than observations is not a defect. It is a claim to test out of sample.**

Source: Measured on this deck's own example four frames on. M. Belkin, D. Hsu, S. Ma, S. Mandal, *PNAS* 116(32), 2019; Y. Liao, X. Ma, A. Neuhierl, Z. Shi, "Does noise hurt economic forecasts?", 2023.

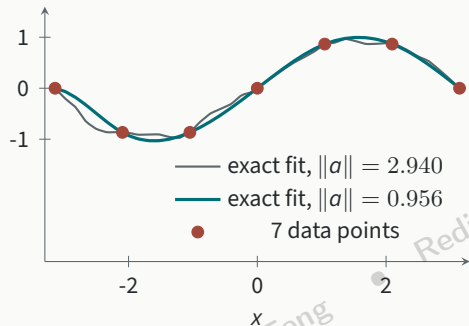
Why Gradient Descent Picks That Fit



- ▶ Linear in the parameters: every gradient $\nabla_a L = X^\top (Xa - y)$, and so every step, lies in the **row space** of X .
- ▶ From $a = 0$ the iterate never leaves it, and one exact fit alone lies there: the smallest $\|a\|_2$. **The optimiser cannot move in directions the data never mention.**
- ▶ $\|\cdot\|_2$ is the selector, Euclidean because the steps are — not a smoothness claim. Multiplicative updates select ℓ_1 instead: the algorithm is part of the estimator.
- ▶ **With $p > n$ the algorithm picks the fit. Whether it generalises is empirical, not a theorem.**

Source: The min-norm fact above is elementary: the derivation is its proof. S. Gunasekar, J. Lee, D. Soudry, N. Srebro, “Characterizing implicit bias in terms of optimization geometry”, ICML 2018.

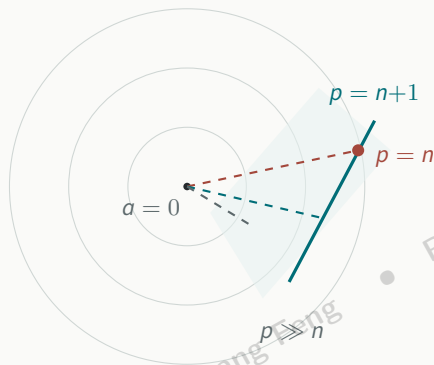
Implicit Regularization, Stated Precisely



- ▶ Seven points of $\sin x$, 41 fixed features: the coefficients $a \in \mathbb{R}^{41}$ are not pinned down, and infinitely many fits are exact.
- ▶ Gradient descent from zero converges to the fit of **smallest Euclidean norm** (teal). No penalty was written.
- ▶ **The optimiser picks among exact fits. For deep networks: intuition, not a theorem.**

Source: Session 2 lab notebook, block B3. Soudry et al., *JMLR* 19, 2018 (the classification analogue: the iterate's *direction* converges to the maximum margin); Hastie et al., *Ann. Statist.* 50(2), 2022 (risk at $p > n$).

Why the Norm Moves: It Is a Minimum over a Set

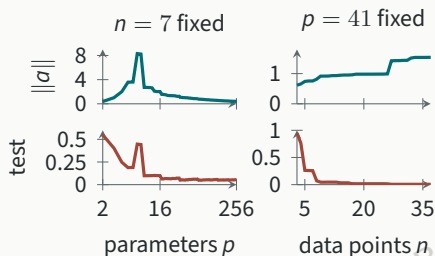


Dashed: the smallest member of each set.

Source: Both directions follow from one argument; both are checked in the Session 2 lab notebook, block B3.

- ▶ $\|a\|$ is the *smallest* coefficient vector that fits. It is a minimum over a set, so whatever changes the set changes it.
- ▶ **More parameters** give more ways to fit: the set grows, so its minimum falls. **More data** give more to satisfy: the set shrinks, so its minimum rises.
- ▶ At $p = n$ the set is a *single point* — nothing to choose — and that is the peak. Below $p = n$ nothing fits exactly at all.

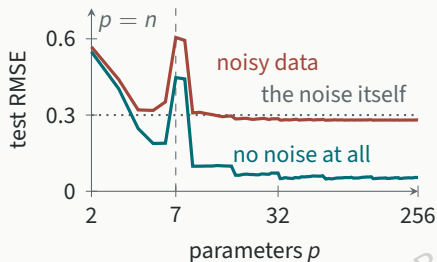
Two Levers, Measured



- **More p (left):** below $p = n$ nothing fits exactly; at $p = n$ the single fit costs 8.3; above it the norm falls to 0.4.
- **More n (right):** $0.62 \rightarrow 1.54$, and every fit is still exact.
- Test error follows neither: past $p = n$ it drops — the optimiser may now *choose* — then stalls near $p = 65$. More data never stops helping, $0.95 \rightarrow 0.006$.
- **The norm falls with the lever that need not help, and rises with the one that always does.**

Source: Session 2 lab notebook, block B3. Nested hinges and nested data points, so each sweep adds to what came before.

Is That Just Overfitting?



- ▶ No: the training error is **zero for every** $p \geq n$ **on both curves**, so a loss below the noise floor cannot tell them apart.
- ▶ **Overfitting** drags the model onto noise. Here every candidate ties at zero; only the tie-break differs.
- ▶ With $y = \sin x$ exactly there is nothing to overfit, yet $p = n$ gives 0.45 against 0.19 at $p = 6$. With noise the two stack and the peak rises to 0.61.
- ▶ **Overfitting is cured with less capacity; this, with a better tie-break or more data.**

Source: Session 2 lab notebook, block B3; $n = 7$, noise s.d. 0.30, mean over 200 draws.

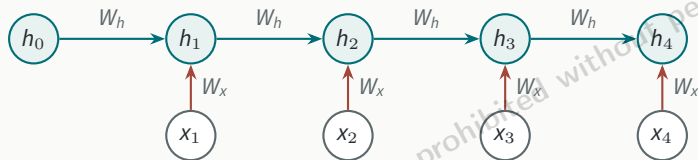
Session 2.6

Order, and Its Absence

Sequences, and sets of agents where names do not matter



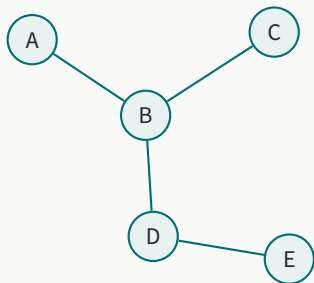
Recurrent Networks: the Hidden State Is a State Variable



- ▶ A **recurrent network** reads a sequence one date at a time: $h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$, the same W_h, W_x at every date.
- ▶ The **hidden state** h_t summarises the history: a state-space model with an estimated transition.
- ▶ Going back T dates multiplies by W_h^T , which vanishes or explodes. An LSTM's gates make persistence depend on the state; an AR(1) has a constant ρ .
- ▶ **The constraint: one law of motion at every date.**

Source: S. Hochreiter and J. Schmidhuber, "Long short-term memory", *Neural Computation* 9(8), 1997; Miao & Feng, §9.6.

Graph Networks: Permutation Invariance Is an Economic Restriction



five banks; an edge is lending

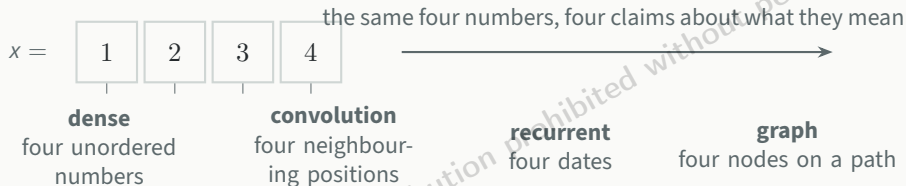
- ▶ A **graph network** updates each node from itself and its neighbours' average, with one pair of matrices for all nodes:
$$h'_v = \sigma(W_s h_v + W_n \bar{h}_{N(v)} + b).$$
- ▶ Neighbours have no order, so the aggregate is a sum or a mean: **permutation invariance**. Renumbering nodes cannot change the answer.
- ▶ **Heterogeneous-agent models impose the same: prices depend on the distribution, not on who is who (Session 4).**

Source: T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks", ICLR 2017; Miao & Feng, §9.6.

Takeaways

- ▶ **An architecture is a prior about the input.** True, it saves parameters and data. False, it is bias.
- ▶ **A convolution is a dense layer after two constraints:** local, and the same filter everywhere.
- ▶ **Sign constraints on W buy a guarantee and pay for it in fit:** 7.8 times the RMSE on the convexity test, and a harder search — not a smaller class.
- ▶ **Shape is the constraint an economist can usually justify,** because theory delivers monotonicity and convexity before any data do.
- ▶ **More parameters than observations is not overfitting:** every candidate ties at zero training error, and only the tie-break differs.
- ▶ **Both halves of this deck are the same question: what decides the answer when the data do not?**

Appendix: One Input, Read Four Ways



- ▶ Every layer below is the same $\sigma(Wx + b)$. They differ only in which entries of W are free, which are fixed at zero, and which are forced to be equal.
- ▶ Weights are chosen here, not trained, so every number on the next two pages can be checked by hand.
- ▶ **The input never changes. The architecture is the claim about what its four numbers are.**

Appendix: Four Layers, Four Answers

	Rule	Output	Free
dense	$w = (\frac{1}{2}, -\frac{1}{2}, \frac{1}{2}, -\frac{1}{2})$	-1	5
convolution	filter (1, -1), slid	(-1, -1, -1)	3
recurrent	$h_t = \frac{1}{2}h_{t-1} + x_t$	(1, 2.5, 4.25, 6.125)	3
graph	$h'_v = x_v + \frac{1}{2}\bar{x}_{N(v)}$	(2, 3, 4.5, 5.5)	3

1	-1	0	0
0	1	-1	0
0	0	1	-1

the convolution's W :
 3×4 , three numbers

- ▶ Dense returns one number, the convolution three, the recurrent and graph layers four: the constraint changes the *shape* of the answer, not only the count of parameters.
- ▶ The convolution's W is written out on the right. Nothing was added to the dense layer — entries were set to zero and the survivors tied together.
- ▶ **Three free numbers do the work of five, and only because the tying is true of the input.**

Source: Every value asserted against the arithmetic in the Session 2 lab notebook, block B.

Appendix: What Each One Refuses to Notice

input	1	2	3	4	convolution: $(-1, -1, -1)$	the dense layer, with its weights relabelled the same way, is unchanged
shuffled	3	1	4	2	convolution: $(2, -3, 2)$	

- ▶ **Shuffle the input.** The convolution's answer is destroyed: it assumed positions 1 and 2 were neighbours, and now they are not. The dense layer never made that claim, so it learns the shuffled problem just as well — the digit slide's point, as arithmetic.
- ▶ **Relabel the graph's nodes** $A B C D$ as $D C B A$: the output becomes $(5.5, 4.5, 3, 2)$, the same four numbers in the new order, and a sum readout stays 15 either way. That is permutation invariance, as a number.
- ▶ The recurrent layer weights the past geometrically, so any reordering of the dates changes the answer: order is the whole of its claim.
- ▶ **A constraint is not a saving. It is a statement about the input that can be wrong.**